

資料探勘 Data Mining

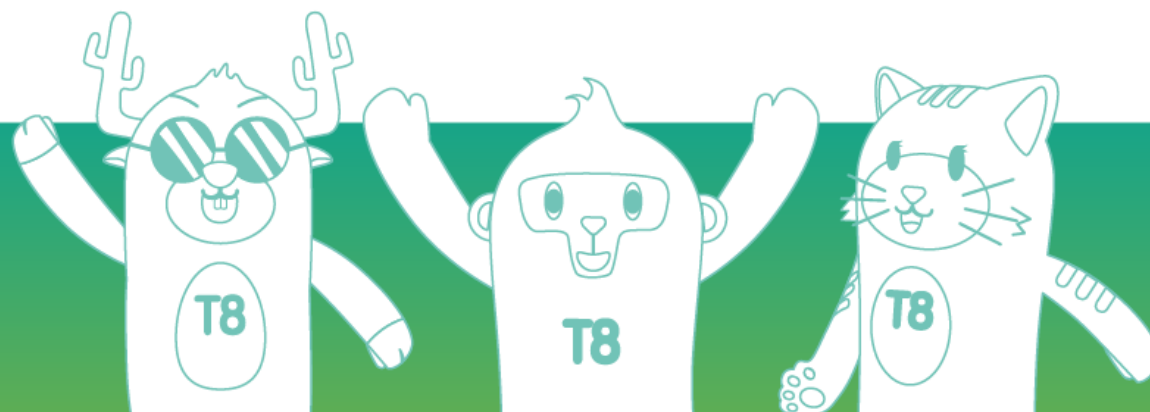
■ 授課講師 陳少君 Paul

■ 教材編寫 陳少君 Paul

緯育 *TibaMe*

即學・即戰・即就業

<https://www.tibame.com/>



課程規劃 與範疇

學習目標：

- 了解本課程的目的
- 了解本課程的 12 個模組

課程規劃 及大綱

課程目的

- 經由影片的講解和範例，以及使用 Python 語言的實作，以由淺入深的方式讓大家逐步了解**資料探勘的原理、步驟以及應用**。

學習步驟

- 從**數據輸入**開始，學習對**資料的清洗**。一旦清洗儲存完成，就可以開始著手分析，將分析結果以清晰易懂的圖表或圖形在螢幕或報表上展示。

課程內容

- 在分析方面，我們會學到商場上最熱門的關聯性分析，以及在 Numpy，Pandas，Matplotlib 等程式庫實作後，學到基本的迴歸預測，資料分類以及分群的原理與與技巧。

學習環境一

- 使用 Google Colab，需有 Google Account 與 Google Drive。

學習環境二

- 也可以下載並安裝 Anaconda，包括 Python 3 和 Jupyter，在自己環境內編譯執行。
- 需考量自身電腦是否有足夠 GPU / TPU 運算能力。

學習環境三

- 有些課程範例會放在 Kaggle.com 上面，若想在 Kaggle 直接體驗，建議建立 Kaggle 帳號。

• <u>課程規劃與範疇</u>	• <u>學習工具與準備</u>	
1. <u>資料探勘簡介</u>	5. <u>資料探索與視覺化</u>	9. <u>ndarray 合併與轉換</u>
2. <u>載入資料</u>	6. <u>關聯分析</u>	10. <u>Pandas 物件運算</u>
3. <u>資料清洗與轉換 (I)</u>	7. <u>Numpy 套件</u>	11. <u>Matplotlib 套件</u>
4. <u>資料清洗與轉換 (II)</u>	8. <u>ndarray 操作</u>	12. <u>更多資料探勘演算法簡介</u>

學習目標：

- 認識甚麼是資料探勘
- 認識其背景與應用

Module 1.

資料探勘簡介

資料探勘簡介

資料探勘 (Data Mining)

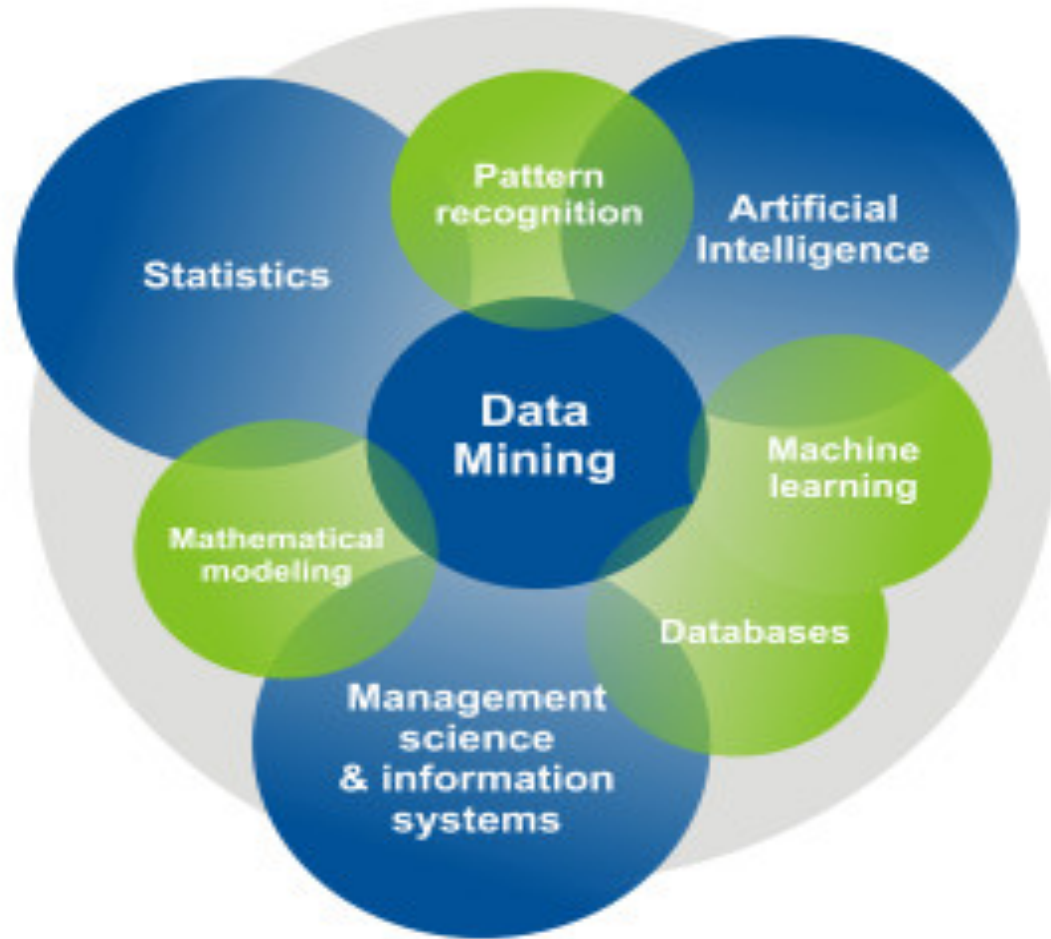
- 是一個跨學科的電腦科學分支。它是用人工智慧、機器學習、統計學和資料庫的交叉方法在相對較大型的資料集中發現模式的計算過程。

資料探勘的目標

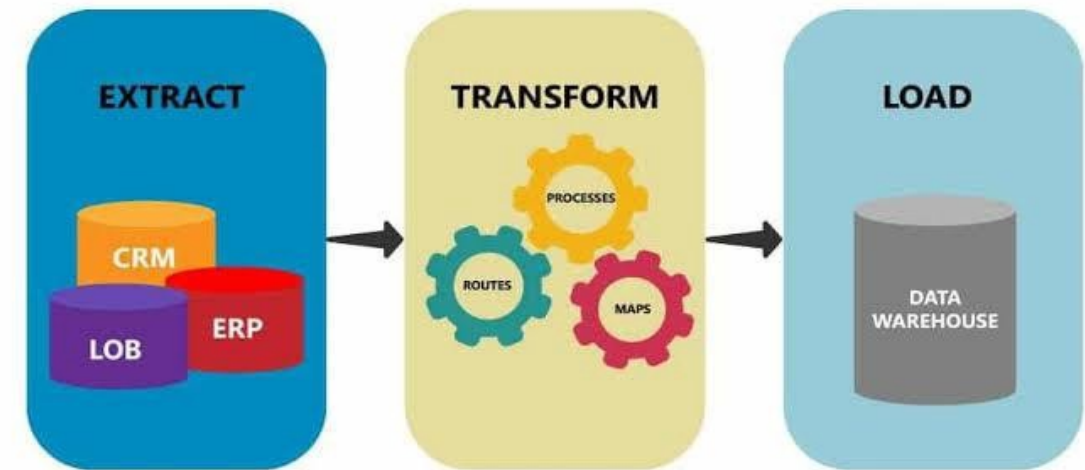
- 是從一個資料集中提取資訊，並將其轉換成可理解的結構，以進一步使用。除了原始分析步驟，它還涉及資料預處理、模型與推斷方面考量，以及發現結構、視覺化等後處理。

資料探勘的範疇

- 資料探勘是「資料庫知識發現」(Knowledge-Discovery in Databases, KDD) 的分析步驟，本質上屬於機器學習的範疇。



- 用到資料庫和資訊管理的知識
- 需要統計分析與數學
- 基礎工作是資料的準備與清洗 (ETL)



ETL - Extract, Transform, Load

Pic from: <http://frontender.com/blog/enablers/data-mining/>

Pic from: <https://medium.com/datadriveninvestor>

學習目標：

- 了解 ETL
- 認識資料儲存
- 認識資料分析與視覺化

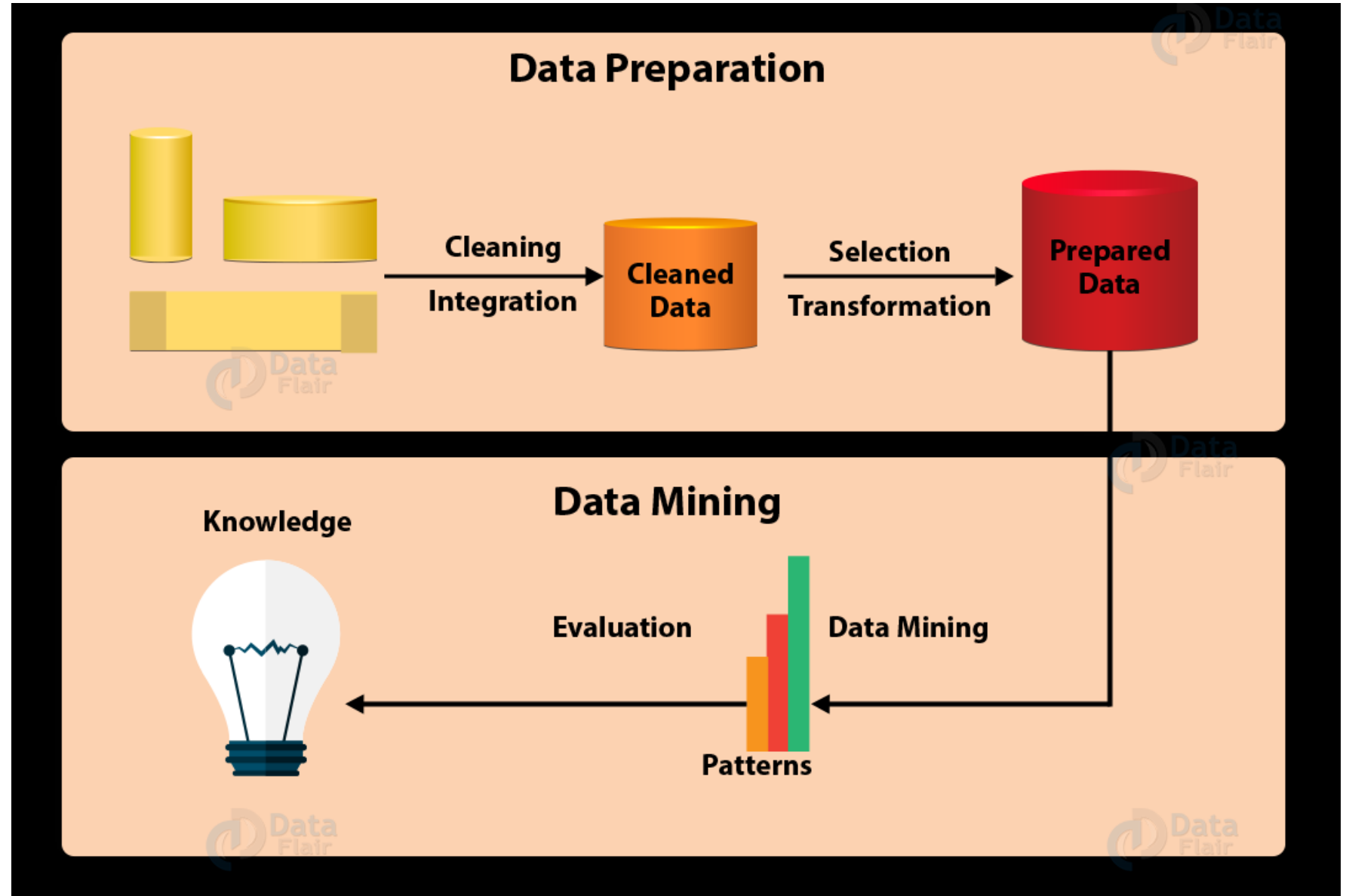
Module 1.

資料探勘簡介

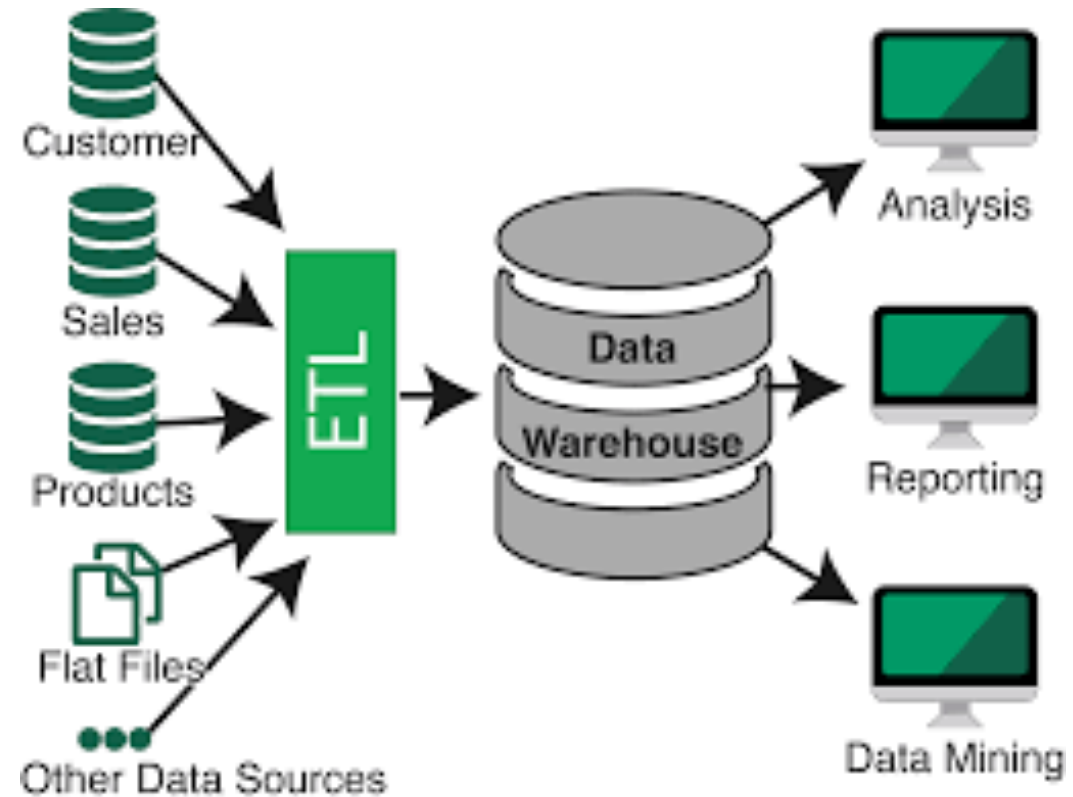
資料探勘程序

資料庫知識發現
(KDD) 過程通常定
義為以下階段：

- (1) 選擇
- (2) 預處理
- (3) 變換
- (4) 資料探勘
- (5) 解釋/評估

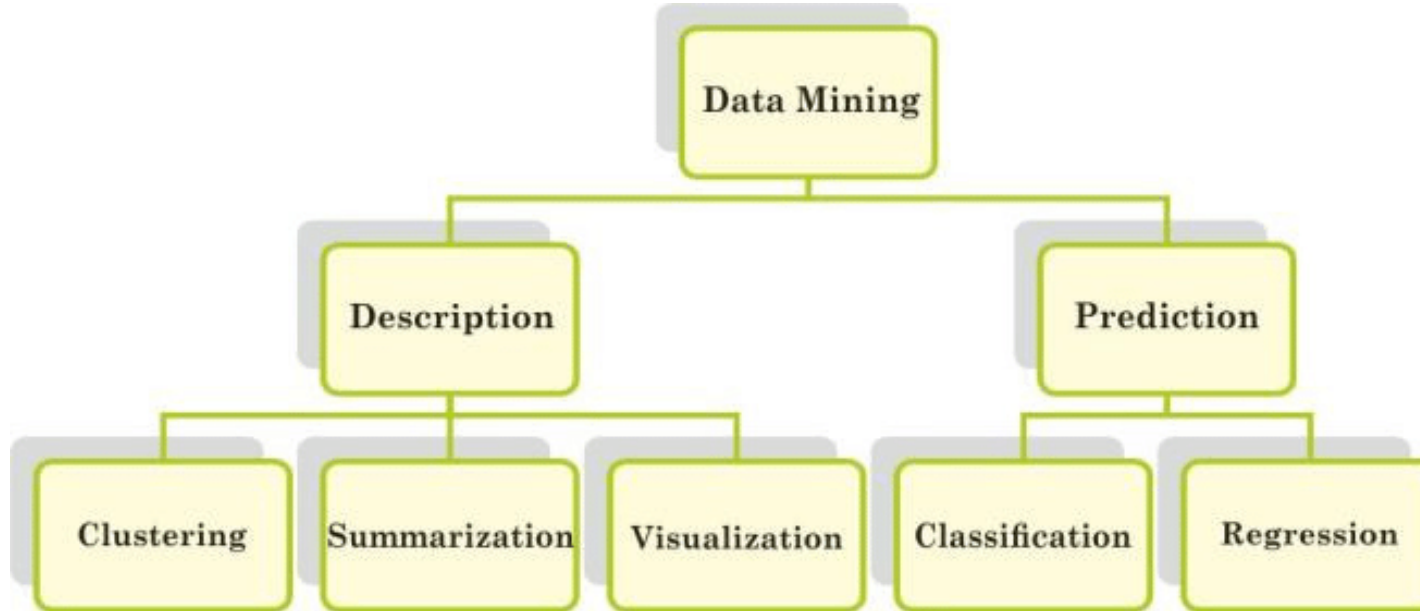


- 在資料探勘建模分析之前，需要經由 ETL（extract-transform-load）將資料準備好
- 分析完成之後，將統計分析以圖表方式呈現報告給相關決策者，俾作成正確決定，進而提高整體生產力



Pic from: <https://data-flair.training/blogs/data-mining-process/>

資料探勘包括預測型 (Prediction) 分析和描述型 (Description) 分析



Pic from: https://www.researchgate.net/figure/Basic-data-mining-models_fig3_308698620

學習目標：

- 認識工具 Pandas, NumPy
- 認識 Matplotlib

Module 1.

資料探勘簡介

資料探勘工具

Data Mining 的幾個開源程式庫或套件：

- ETL 部分有 Pandas
- 運算分析建模有 Numpy, SKLearn
- 視覺化有 Matplotlib
- 基礎部分有 Numpy

Pandas

是 Python 的一個數據分析 lib，提供高效能、簡易使用的資料格式（Data Frame）：

1. 異質數據的讀取、轉換和載入，
2. 提供 Series 與 DataFrame，
3. 快速地進行資料的前處理，如資料補值，空值去除或取代等，
4. 更多的輸入來源及輸出整合性。

Matplotlib

- Matplotlib 是利用 Python 所實作的繪圖套件，其中包含兩個最重要的模組 — pylab 和 pyplot。

SKLearn

- scikit-learn，又寫作 sklearn，是一個開源的基於 Python 語言的機器學習工具包。它通過 NumPy, SciPy 和 Matplotlib 等 Python 數值計算的庫實現高效的演算法應用，並且涵蓋了幾乎所有主流機器學習演算法。

NumPy 簡介

是 Python 在進行科學運算時一個基礎的 Package，同時也是 Python 核心的 library：

1. 提供高效能多維陣列數學函式庫
2. 提供線性代數及傅立葉轉換的功能。
3. 利用 NumPy Array (ND Array) 替代 Python List，提高運算速度與統一性。
4. 可定義任意的數據型態（Data Type），整合多種資料庫接口。

學習目標：

- 認識 CSV, Excel 格式
- 學會輸入與處理

Module 2.

載入資料

處理 CSV, EXCEL
格式資料

Pandas 提供 CSV，Excel 輸入功能，作法如下
(df 是 Pandas DataFrame)：

讀入 CSV

目的	作法
read csv	<code>df=pandas.read_csv("scores.csv")</code>
from certain folder	<code>df=pandas.read_csv("C:\\folder\\sub_folder\\scores.csv")</code>

Pandas 提供 CSV，Excel 輸入功能，作法如下
(df 是 Pandas DataFrame)：

讀入 Excel

目的	作法
read Excel	<code>df = pandas.read_excel("scores.xlsx")</code>
from certain folder	<code>df=pandas.read_excel("C:\\folder\\sub_folder\\scores.xlsx")</code>
from certain sheet	<code>df=pandas.read_excel("C:\\folder\\sub_folder\\scores.xlsx",sheet_name="Sheet1")</code>
from multiple sheets (assume same schema)	<code>df=pandas.read_excel("C:\\folder\\sub_folder\\scores.xlsx",sheet_name=["Sheet1","Sheet2"])</code>

Column Selection : Pandas Data Frame 可選擇讀入一或多個 Series (Column) , 作法如下 :

目的	作法
起始	<pre>import pandas as pd melbourne_file_path = '../input/melbourne-housing-snapshot/melb_data.csv' melbourne_data = pd.read_csv(melbourne_file_path)</pre>
印出所有欄位	<pre>print(melbourne_data.columns)</pre>
讀取 Price 欄	<pre>melbourne_price_data = melbourne_data.Price print(melbourne_price_data.head())</pre>
任取兩欄	<pre>columns_of_interest = ['Landsize', 'BuildingArea'] two_columns_of_data = melbourne_data[columns_of_interest] two_columns_of_data.describe()</pre>

Row Selection : Pandas Data Frame 也可根據 Column Value 選擇讀入 Rows，作法如下：

目的	作法
起始	<pre>import pandas as pd data_url = 'http://bit.ly/2cLzoxH'</pre>
將網址內容讀成 Data Frame	<pre>gapminder = pd.read_csv(data_url)</pre>
設定選擇判準	<pre>is_2002 = gapminder['year']==2002</pre>
DF 只讀 year==2002 資料	<pre>gapminder_2002 = gapminder[is_2002]</pre>
Pandas 鏈結 (Chaining)	<pre>gapminder_2002 = gapminder[gapminder.year.eq(2002)]</pre>

Row Selection : Pandas Data Frame 也可根據 Column Value 選擇讀入 Rows，作法如下：

目的	作法
#過濾 Null 資料	<code>gapminder_no_NA = gapminder[gapminder.year.notnull()]</code>
#只選擇 years 在 list [1952, 2007] 中的 rows	<code>years = [1952, 2007] gapminder.year.isin(years)</code>
#Unique value	<code>gapminder_years.year.unique()</code>
#多重條件	<code>continents = ['Asia','Africa', 'Americas', 'Europe'] gapminder[~gapminder.continent.isin(continen ts) & gapminder.year.isin(years)]</code>

Module 2.

載入資料

學習目標：

- 認識 JSON 與 XML 格式
- 學會輸入與儲存

處理 JSON, XML 格式資料

JSON：讀入寫出 JSON 時要考慮幾種

orientation orient = 'split' , 'index' , 'records' , 'table'

JSON Orient	JSON 格式
split	dict like { 'index' -> [index], 'columns' -> [columns], 'data' -> [values]}
index	dict like {index -> {column -> value}}
records	list like [{column -> value}, ... , {column -> value}]
table	dict like { 'schema' : {schema}, 'data' : {data}}
columns	dict like {column -> {index -> value}}
values	just the values array


```
#'split' : dict like {index -> [index], columns -> [columns], data -> [values]}
```

```
df = pd.DataFrame([[ 'a', 'b'], [ 'c', 'd']],  
                  index=['row 1', 'row 2'],  
                  columns=['col 1', 'col 2' ])
```

```
df.to_json(orient='split')
```

```
'{"columns":["col 1","col 2"],  
  "index":["row 1","row 2"],  
  "data":[["a","b"],["c","d"]}]'
```

```
pd.read_json(_, orient='split')
```

	col 1	col 2
row 1	a	b
row 2	c	d

```
#'index' : dict like {index -> {column -> value}}
```

```
df.to_json(orient='index')
```

```
'{"row 1":{"col 1":"a","col 2":"b"},"row 2":{"col 1":"c","col 2":"d"}}'
```

```
pd.read_json(_, orient='index')
```

```
col 1 col 2
row 1    a    b
row 2    c    d
```

```
#'records' : list like [{column -> value}, ... , {column -> value}]
```

```
df.to_json(orient='records')
```

```
'[{"col 1":"a","col 2":"b"}, {"col 1":"c","col 2":"d"}]'
```

```
pd.read_json(_, orient='records')
```

```
col 1 col 2
0    a    b
1    c    d
```

```
# orient = 'table'
```

```
df.to_json(orient='table')
```

```
'{"schema": {"fields": [{"name": "index", "type": "string"},  
                        {"name": "col 1", "type": "string"},  
                        {"name": "col 2", "type": "string"}],  
            "primaryKey": "index",  
            "pandas_version": "0.20.0"},  
  "data": [{"index": "row 1", "col 1": "a", "col 2": "b"},  
           {"index": "row 2", "col 1": "c", "col 2": "d"}]}'
```

```
pd.read_json(_, orient='table')
```

	col 1	col 2
row 1	a	b
row 2	c	d

XML：沒有現成的函數將 XML 讀入 Pandas DataFrame，可利用 Python 的 etree module 自己開發。

說明	程式碼
加載模組	<pre>Import pandas as pd Import xml.etree.ElementTree as etree</pre>
#假設我們有一個 employee XML file 在 root (即 employee) 之下只有三平行欄位 ["name", "email", "age"]	<pre>tree = etree.parse("employee.xml") root = tree.getroot() columns = ["name", "email", "age"] datatframe = pd.DataFrame(columns = columns)</pre>
For 迴圈讀入各員工資料	<pre>for node in root: name = node.attrib.get("name") mail = node.find("email").text if node is not None else None age = node.find("age").text if node is not None else None datatframe = datatframe.append(pd.Series([name, mail, age], index = columns), ignore_index = True)</pre>

學習目標：

- 認識資料庫的特色
- 認識 SQL 與 NoSQL

Module 2.

載入資料

讀取資料庫資料

讀取資料庫資料可用：

pandas.read_sql, pandas.read_query, pandas.read_sql_table

說明	程式碼
加載模組	<pre>import pandas as pd import numpy as np from sqlalchemy import create_engine</pre>
SQLite 引擎初始	<pre>engine = create_engine('sqlite://', echo=False)</pre>
將一筆時間資料寫入SQLite	<pre>df = pd.DataFrame({'A': pd.date_range('2013-01-01 09:00:00', periods=3), 'B': np.arange(3.0)}) df.to_sql('test_datetime', con=engine)</pre>
資料讀回	<pre>result = pd.read_sql_table('test_datetime', con=engine)</pre>
捨去行方向索引	<pre>result = result.drop('index', axis=1)</pre>

學習目標：

- 認識篩選資料的原則與方法

Module 3.

資料清洗與轉換 (I)

資料篩選

資料篩選的用途：節省儲存空間，提升運算效率。

對 Data Frame 或 Series 篩選的過濾器：

說明	程式碼
#假設 employees 是一個 Data Frame，有 Gender, Salary, Team, Start Date 等 series	fliter = (employees["Gender"] == "Female")
#只選取女性	employees[fliter]

說明	程式碼
#只選取高薪的	<pre>employees["Salary"] > 200000 employees[employees["Salary"] > 200000]</pre>
#條件組合	<pre>mask1 = employees["Team"] != "Marketing" mask2 = employees["Start Date"] < "1990-01-01 " employees[(mask1 & mask2)]</pre>
# isin, isnull, between 等其他篩選條件	<pre>mask = employees["Team"].isin(["Legal","Sales", "Product"]) mask = employees["Team"].notnull() employees[employees["Start Date"].between("2009-11-10","2010-01- 20")]</pre>

學習目標：

- 了解時序資料的特色
- 學會處理時序資料

Module 3.

資料清洗與轉換 (I)

文字與時間資料處理

Pandas 處理文字的兩種方法：

(1) 當成 Object

(2) 當成字串

說明	程式碼	輸出
當成Object	<code>pd.Series(['a', 'b', 'c'])</code>	<pre>0 a 1 b 2 c dtype: object</pre>
當成字串	<code>pd.Series(['a', 'b', 'c'], dtype="string")</code>	<pre>0 a 1 b 2 c dtype: string</pre>
也可用 <code>astype</code> 改來改去	<code>s.astype("string")</code>	

說明	程式碼	輸出
字串分裂	<pre>s2 = pd.Series(['a_b_c', 'c_d_e', np.nan, 'f_g_h'], dtype="string") s2.str.split('_')</pre>	<pre>0 [a, b, c] 1 [c, d, e] 2 <NA> 3 [f, g, h] dtype: object</pre>
字串連結	<pre>t = pd.Series(['a', 'b', np.nan, 'd'], dtype="string") t.str.cat(sep=',')</pre>	'a,b,d'
	<pre>t.str.cat(sep=',', na_rep='-')</pre>	'a,b,-,d'

Pandas 處理時間序列：能整合許多時序資料成 Date time, Time Delta, Time Span, Time Difference

說明	程式碼	輸出
以日為單位創造三個時間	<code>pd.Series(range(3), index=pd.date_range('2000', freq='D', periods=3))</code>	2000-01-01 0 2000-01-02 1 2000-01-03 2 Freq: D, dtype: int64
時間算數，Add 1 day	<code>saturday = friday + pd.Timedelta('1 day') saturday.day_name()</code>	'Saturday'
將 Series 頻率調成 2 小時	<code>idx = pd.date_range('2018-01-01', periods=5, freq='H') ts = pd.Series(range(len(idx)), index=idx) ts</code>	2018-01-01 00:00:00 0 2018-01-01 01:00:00 1 2018-01-01 02:00:00 2 2018-01-01 03:00:00 3 2018-01-01 04:00:00 4 Freq: H, dtype: int64
	<code>ts.resample('2H').mean()</code>	2018-01-01 00:00:00 0.5 2018-01-01 02:00:00 2.5 2018-01-01 04:00:00 4.0 Freq: 2H, dtype: float64

說明	程式碼	輸出
Time Stamp，某時間點	<code>pd.Timestamp(datetime.datetime(2012, 5, 1))</code>	<code>Timestamp('2012-05-01 00:00:00')</code>
Time Span，時間長度， 從2012-5-1每天	<code>pd.Period('2012-05', freq='D')</code>	<code>Period('2012-05-01', 'D')</code>
Time Stamp 自動變成 DF 裡的 Index	<code>dates = [pd.Timestamp('2012-05-01'), pd.Timestamp('2012-05-02'), pd.Timestamp('2012-05-03')] ts = pd.Series(np.random.randn(3), dates) type(ts.index)</code>	<code>pandas.core.indexes.datetimes.DatetimeIndex</code>
用 Timestamp 當 Index	<code>ts.index</code>	<code>DatetimeIndex(['2012-05-01', '2012-05-02', '2012-05-03'], dtype='datetime64[ns]', freq=None)</code>
	<code>ts</code>	<code>2012-05-01 0.469112 2012-05-02 -0.282863 2012-05-03 -1.509059 dtype: float64</code>

學習目標：

- 了解遺失值
- 認識偵測與補償

Module 3.

資料清洗與轉換 (I)

遺失值偵測與填補

- Pandas 認為 None 和 NaN (Not a number) 都是遺失值，可以下列方法去偵測：[isnull\(\)](#)，[notnull\(\)](#)
- 填補可用 [fillna\(\)](#)，[replace\(\)](#)，[interpolate\(\)](#)

遺失值偵測說明	程式碼
importing pandas as pd	import pandas as pd
importing numpy as np	import numpy as np
dictionary of lists	dict = {'First Score':[100, 90, np.nan, 95], 'Second Score': [30, 45, 56, np.nan], 'Third Score':[np.nan, 40, 80, 98]}
creating a dataframe from list	df = pd.DataFrame(dict)
using isnull() function	df.isnull()

#產生下圖



	First Score	Second Score	Third Score
0	False	False	True
1	False	False	False
2	True	False	False
3	False	True	False

- Pandas 填補遺失值用 [fillna\(\)](#) , [replace\(\)](#) , [interpolate\(\)](#)

用fillna(0) 來填補遺失值	程式碼
importing pandas as pd	import pandas as pd
importing numpy as np	import numpy as np
dictionary of lists	dict = {'First Score':[100, 90, np.nan, 95], 'Second Score': [30, 45, 56, np.nan], 'Third Score':[np.nan, 40, 80, 98]}
creating a dataframe from dictionary	df = pd.DataFrame(dict)
filling missing value using fillna()	df.fillna(0)



	First Score	Second Score	Third Score
0	100.0	30.0	0.0
1	90.0	45.0	40.0
2	0.0	56.0	80.0
3	95.0	0.0	98.0

Pandas 填補遺失值用 [fillna\(\)](#) , [replace\(\)](#) , [interpolate\(\)](#)

說明	程式碼
用fillna(method= 'pad')前一個值來填補遺失值 也可用fillna(method= 'bfill')後一個值來填補遺失值	import pandas as pd import numpy as np
dictionary of lists	dict = {'First Score':[100, 90, np.nan, 95], 'Second Score': [30, 45, 56, np.nan], 'Third Score':[np.nan, 40, 80, 98]}
creating a dataframe from dictionary	df = pd.DataFrame(dict)
filling a missing value with previous ones	df.fillna(method ='pad')



	First Score	Second Score	Third Score
0	100.0	30.0	NaN
1	90.0	45.0	40.0
2	90.0	56.0	80.0
3	95.0	56.0	98.0

Df.fillna(method= 'bfill')

	First Score	Second Score	Third Score
0	100.0	30.0	40.0
1	90.0	45.0	40.0
2	95.0	56.0	80.0
3	95.0	NaN	98.0

Pandas 填補遺失值用 [replace\(\)](#) , [interpolate\(\)](#)

說明	程式碼
#replace	df.replace(to_replace = np.nan, value = -99)
	import pandas as pd
# Creating the dataframe	df = pd.DataFrame({"A":[12, 4, 5, None, 1], "B":[None, 2, 54, 3, None], "C":[20, 16, None, 3, 8], "D":[14, 3, None, None, 6]})
# Print the dataframe	df
# to interpolate the missing values	df.interpolate(method = 'linear', limit_direction = 'forward')

	A	B	C	D
0	12.0	NaN	20.0	14.0
1	4.0	2.0	16.0	3.0
2	5.0	54.0	NaN	NaN
3	NaN	3.0	3.0	NaN
4	1.0	NaN	8.0	6.0

↓

	A	B	C	D
0	12.0	NaN	20.0	14.0
1	4.0	2.0	16.0	3.0
2	5.0	54.0	9.5	4.0
3	3.0	3.0	3.0	5.0
4	1.0	3.0	8.0	6.0

學習目標：

- 了解離群值定義
- 學會離群值處理

Module 4.

資料清洗與轉換（II）

離群值處理

離群值 (Outlier) 定義：

在蒐集資料過程因測量方法、人為疏失或實驗誤差等因素，導致出現個別值極度異於其它資料的值。

離群值判準：

- 在 3rd Quartier 之上和 1st Quartier 之下 1.5 倍 interquartile range (IQR) 的資料點。 $(IQR = 3^{\text{rd}} \text{ Quartier} - 1^{\text{st}} \text{ Quartier})$
- 在與平均值之差超過 3 個標準差的資料點

Pandas 離群值處理方法：

- 偵測：自行開發偵測 標準差或 IQR 方式
- 處理：移除或用其他方式選擇性移除或手動處理

離群值偵測：data_1 是 Pandas DF，將與 mean 差 3 z-score 的 Outliers 回傳

```
import numpy as np
import pandas as pd
outliers=[]

def detect_outlier(data_1):
    threshold=3
    mean_1 = np.mean(data_1)
    std_1 =np.std(data_1)

    for y in data_1:
        z_score= (y - mean_1)/std_1
        if np.abs(z_score) > threshold:
            outliers.append(y)
    return outliers
```

離群值移除：df 是 Pandas DF，將整個 DF 取 z-score，取絕對值，然後在每個 column 中看到 (axis = 1，橫向) z-score < 3 的 作成 filter，然後把其他 (>3) 的 row 過濾掉。

```
#calculate z-scores of `df`  
z_scores = stats.zscore(df)  
  
abs_z_scores = np.abs(z_scores)  
#定義 filter entries  
filtered_entries = (abs_z_scores < 3).all(axis=1)  
#執行 filter  
new_df = df[filtered_entries]  
  
print(new_df)
```

學習目標：

- 認識資料格式轉換
- 學會轉換之處理

Module 4.

資料清洗與轉換（II）

資料轉換

資料轉換

是 ETL 之中 T(Transform) 的部分，目的是在分析前的資料清洗（cleaning）和加總（aggregation）。

基本的 Data Transformation 功能

cleaning, duplication, format revision, key restructuring 等。

進階的 Data Transformation 功能

Derivation, Filtering, Joining, Splitting, Data Validation, Summerization, Aggregation, Integration 等。

使用 Pandas 作資料轉換的優劣點

優點

- 被廣泛使用
- 語法簡單直覺
- 與其他 Python 工具，包括視覺化整合完美
- 支援常用資料格式（SQL 資料庫，CSV 檔等）

缺點

- 擴充性差，不適合處理超大資料集

學習目標：

- 了解資料如何合併
- 認識資料重塑方法

Module 4.

資料清洗與轉換（II）

資料合併與重塑

目的：

在資料正確性無誤後，為配合資料倉庫（Data Warehouse）的綱要（Schema）或運作需要而作的轉換。

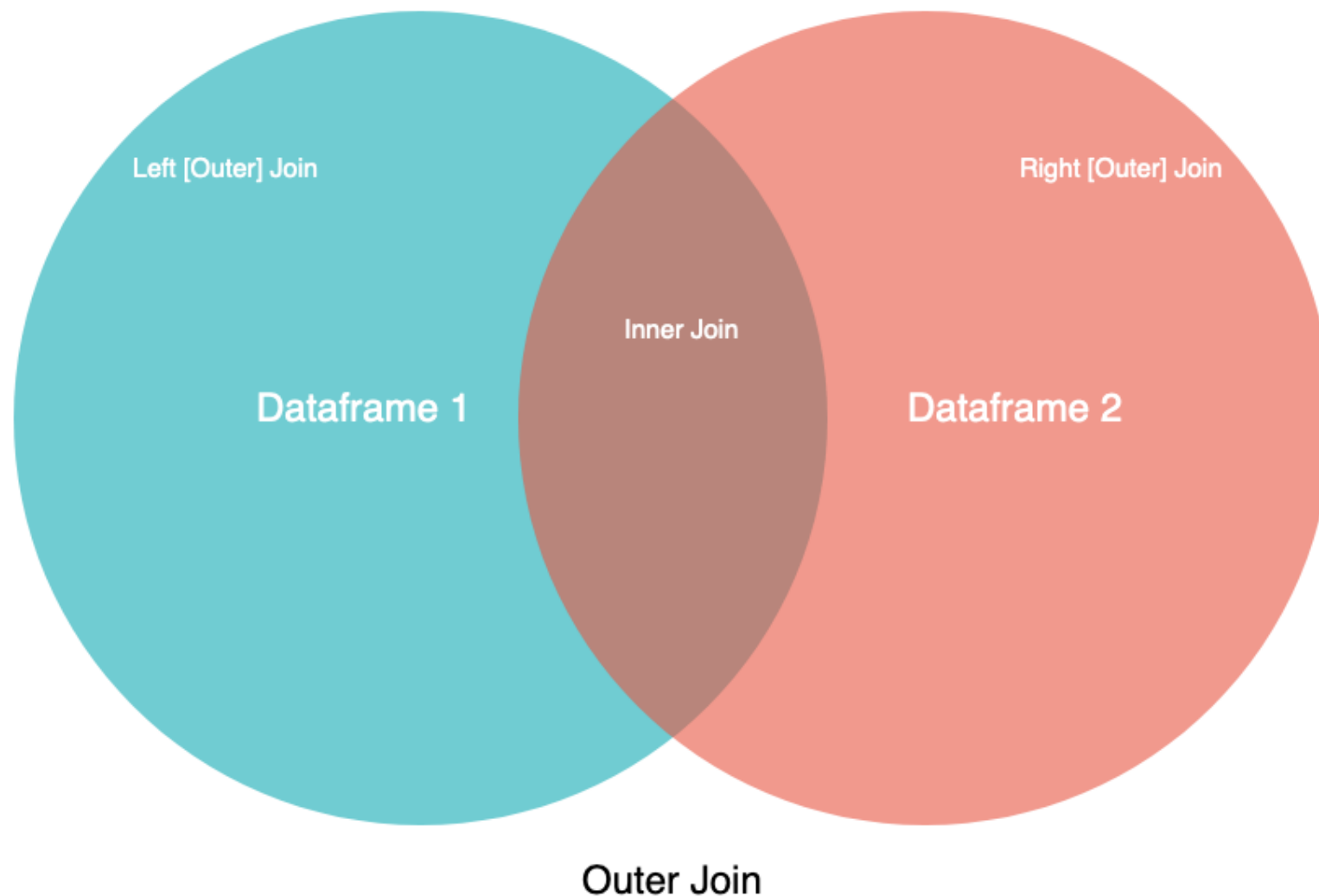
意義：

是將多個資料來源依據共同欄位作成橫向延伸，形成一個更多欄位的 Data Frame / Table；或者將同一來源的多份資料，以及欄位定義相匹配的資料做縱橫向的堆疊或轉軸。

Pandas 提供合併與重塑的功能：

merge()：	合併共同的 columns 或 indices
.join()：	左邊某一個 key column or an index 和右邊 index 合併資料
concat()：	對 DataFrames 的 rows 或 columns 以 index 為主的合併
stack()/unstack()：	將 columns 重塑為 row data ，或 row data 轉成 columns

首先我們要看圖了解 4 種 JOIN：INNER，OUTER，LEFT and RIGHT



Merge :

```
pd.merge(left, right, how= 'inner' , on=None, left_on=None, right_on=None,
left_index=False, right_index=False, sort=True, suffixes=('_x', '_y'), copy=True,
indicator=False, validate=None)
```

Source

	DF A			DF B	
	lkey	value		rkey	value
0	foo	1	0	foo	5
1	bar	2	1	bar	6
2	baz	3	2	qux	7
3	foo	4	3	bar	8

Outer
➡

A.merge(B, left_on='lkey', right_on='rkey',how='outer')

	lkey	value_x	rkey	value_y
0	foo	1	foo	5
1	foo	4	foo	5
2	bar	2	bar	6
3	bar	2	bar	8
4	baz	3	NaN	NaN
5	NaN	NaN	qux	7

Merge (Inner and Left) :

Source

	DF A			DF B	
	lkey	value		rkey	value
0	foo	1	0	foo	5
1	bar	2	1	bar	6
2	baz	3	2	qux	7
3	foo	4	3	bar	8

Inner



A.merge(B, left_on='lkey', right_on='rkey', how= 'inner')

	lkey	value_x	rkey	value_y
0	foo	1	foo	5
1	foo	4	foo	5
2	bar	2	bar	6
3	bar	2	bar	8

A.merge(B, left_on= 'lkey' , right_on='rkey', how= 'left')

Left



	lkey	value_x	rkey	value_y
0	foo	1	foo	5
1	foo	4	foo	5
2	bar	2	bar	6
3	bar	2	bar	8
4	baz	3	NaN	NaN

Join : DataFrame.join(df, on=None, how='left', lsuffix="", rsuffix="", sort=False)

```
left = pd.DataFrame({'A': ['A0', 'A1', 'A2', 'A3'], ....: 'B': ['B0', 'B1', 'B2', 'B3'], ....: 'key': ['K0', 'K1', 'K0', 'K1' ]})
```

```
right = pd.DataFrame({'C': ['C0', 'C1'], ....: 'D': ['D0', 'D1']}, ....: index=['K0', 'K1' ])
```

```
result = left.join(right, on='key')
```

left				right			Result					
	A	B	key		C	D		A	B	key	C	D
0	A0	B0	K0				0	A0	B0	K0	C0	D0
1	A1	B1	K1	K0	C0	D0	1	A1	B1	K1	C1	D1
2	A2	B2	K0	K1	C1	D1	2	A2	B2	K0	C0	D0
3	A3	B3	K1				3	A3	B3	K1	C1	D1

Concat : `pd.concat(objs, axis=0, join='outer', ignore_index=False, keys=None, levels=None, names=None, verify_integrity=False, copy=True)`

舉例：

`frames = [df1, df2, df3]`

`result = pd.concat(frames, keys=['x', 'y', 'z' ])`

結果將形成階層式的索引

`result.loc['y']`

A B C D

4 A4 B4 C4 D4

5 A5 B5 C5 D5

6 A6 B6 C6 D6

7 A7 B7 C7 D7

df1					Result					
	A	B	C	D			A	B	C	D
0	A0	B0	C0	D0	x	0	A0	B0	C0	D0
1	A1	B1	C1	D1	x	1	A1	B1	C1	D1
2	A2	B2	C2	D2	x	2	A2	B2	C2	D2
3	A3	B3	C3	D3	x	3	A3	B3	C3	D3
df2					y	4	A4	B4	C4	D4
	A	B	C	D	y	5	A5	B5	C5	D5
4	A4	B4	C4	D4	y	6	A6	B6	C6	D6
5	A5	B5	C5	D5	y	7	A7	B7	C7	D7
6	A6	B6	C6	D6	z	8	A8	B8	C8	D8
7	A7	B7	C7	D7	z	9	A9	B9	C9	D9
df3					z	10	A10	B10	C10	D10
	A	B	C	D	z	11	A11	B11	C11	D11
8	A8	B8	C8	D8						
9	A9	B9	C9	D9						
10	A10	B10	C10	D10						
11	A11	B11	C11	D11						

Concat 的幾種延伸：

#Outer Join, Axis = 1

```
df4 = pd.DataFrame({'B': ['B2', 'B3', 'B6', 'B7'],
...:                'D': ['D2', 'D3', 'D6', 'D7'],
...:                'F': ['F2', 'F3', 'F6', 'F7']},
...:               index=[2, 3, 6, 7])
```

```
result = pd.concat([df1, df4], axis=1,
sort=False)
```

#Inner Join, Axis = 1:

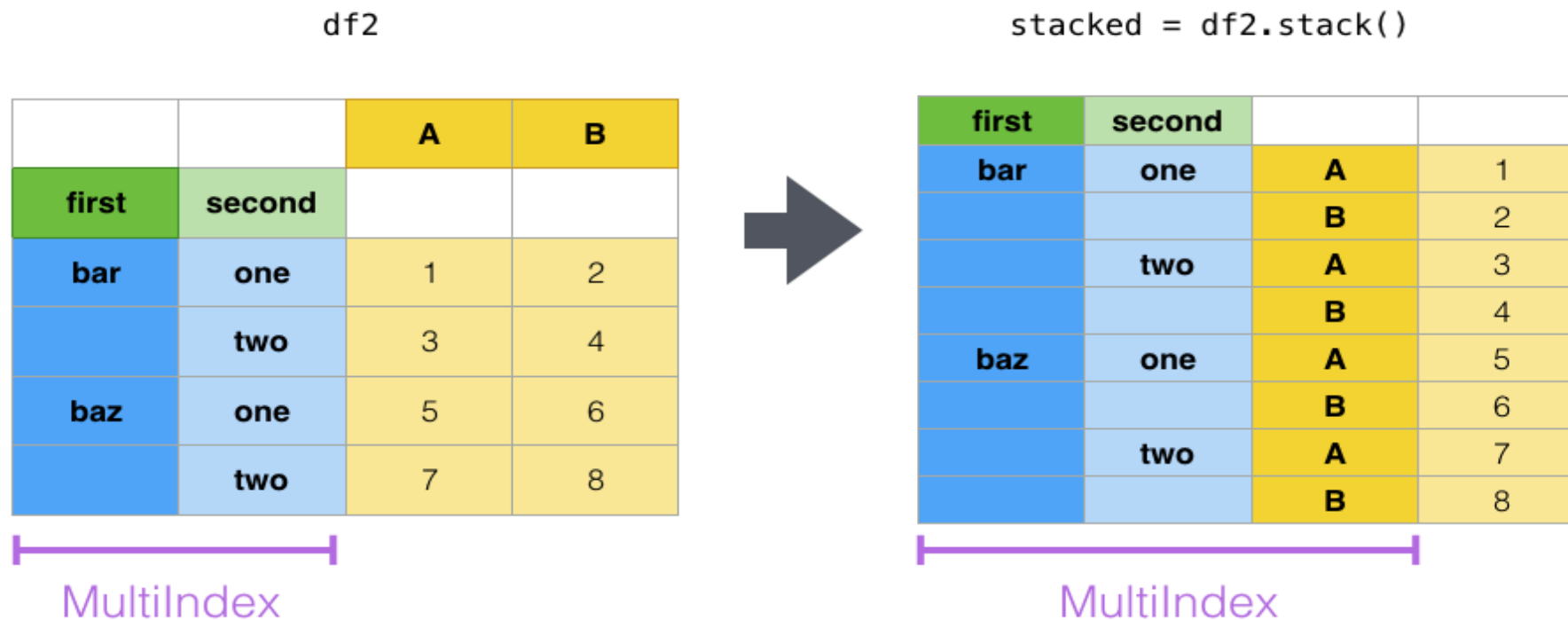
```
result = pd.concat([df1, df4], axis=1,
join='inner')
```

df1					df4				Result							
	A	B	C	D		B	D	F		A	B	C	D	B	D	F
0	A0	B0	C0	D0	2	B2	D2	F2	0	A0	B0	C0	D0	NaN	NaN	NaN
1	A1	B1	C1	D1	3	B3	D3	F3	1	A1	B1	C1	D1	NaN	NaN	NaN
2	A2	B2	C2	D2	6	B6	D6	F6	2	A2	B2	C2	D2	B2	D2	F2
3	A3	B3	C3	D3	7	B7	D7	F7	3	A3	B3	C3	D3	B3	D3	F3
									6	NaN	NaN	NaN	NaN	B6	D6	F6
									7	NaN	NaN	NaN	NaN	B7	D7	F7

df1					df4				Result							
	A	B	C	D		B	D	F		A	B	C	D	B	D	F
0	A0	B0	C0	D0	2	B2	D2	F2	2	A2	B2	C2	D2	B2	D2	F2
1	A1	B1	C1	D1	3	B3	D3	F3	3	A3	B3	C3	D3	B3	D3	F3
2	A2	B2	C2	D2	6	B6	D6	F6								
3	A3	B3	C3	D3	7	B7	D7	F7								

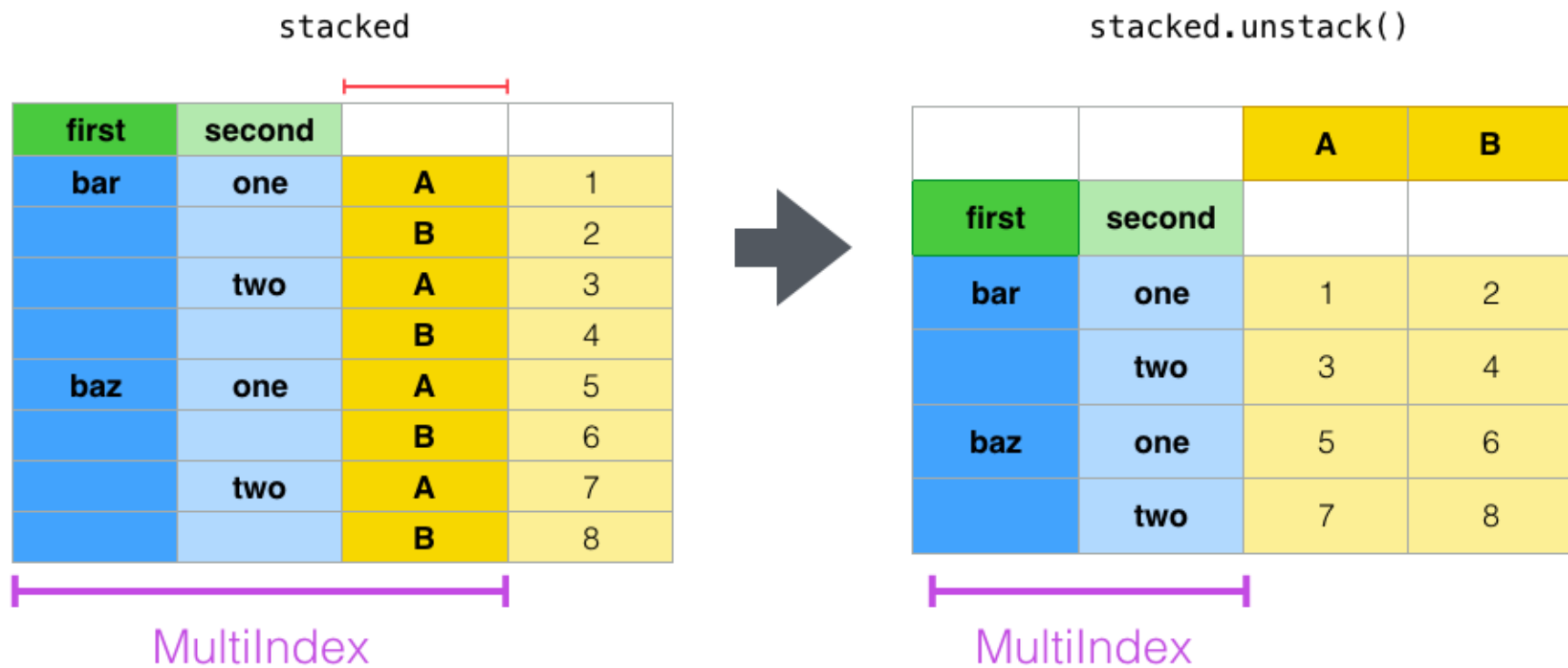
資料重塑 (Reshape) 或轉軸 (Pivot) , Stack 把 Columns 堆疊成 Row (index) data 如下 :

Stack



資料重塑 (Reshape) 或轉軸 (Pivot) , Unstack 把 Row (index) data 展開成 Columns 如下 :

Unstack



學習目標：

- 認識何謂 EDA
- 認識 EDA 探索途徑

Module 5.

資料探索與視覺化

探索性資料分析 (EDA)

How to Explore Data

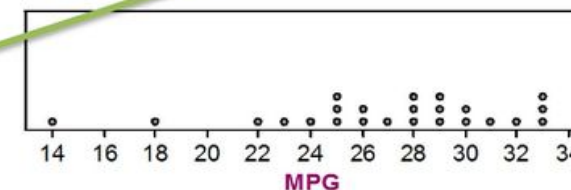
探索性資料分析 (EDA) 是指：

- 發現模式 (discover pattern)
- 檢驗假設 (test hypothesis)
- 檢查假設 (check assumption)

**Examine each variable by itself.
Then study relationships among the variables.**

MODEL	MPG	MODEL	MPG	MODEL	MPG
Acura RL	22	Dodge Avenger	30	Mercedes-Benz E350	24
Audi A6 Quattro	23	Hyundai Elantra	33	Mercury Milan	29
Bentley Arnage	14	Jaguar XF	25	Mitsubishi Galant	27
BMW 528i	28	Kia Optima	32	Nissan Maxima	26
Buick Lacrosse	28	Lexus GS 350	26	Rolls Royce Phantom	18
Cadillac CTS	25	Lincoln MKZ	28	Saturn Aura	33
Chevrolet Malibu	33	Mazda 6	29	Toyota Camry	31
Chrysler Sebring	30	Mercedes-Benz E350	24	Volkswagen Passat	29

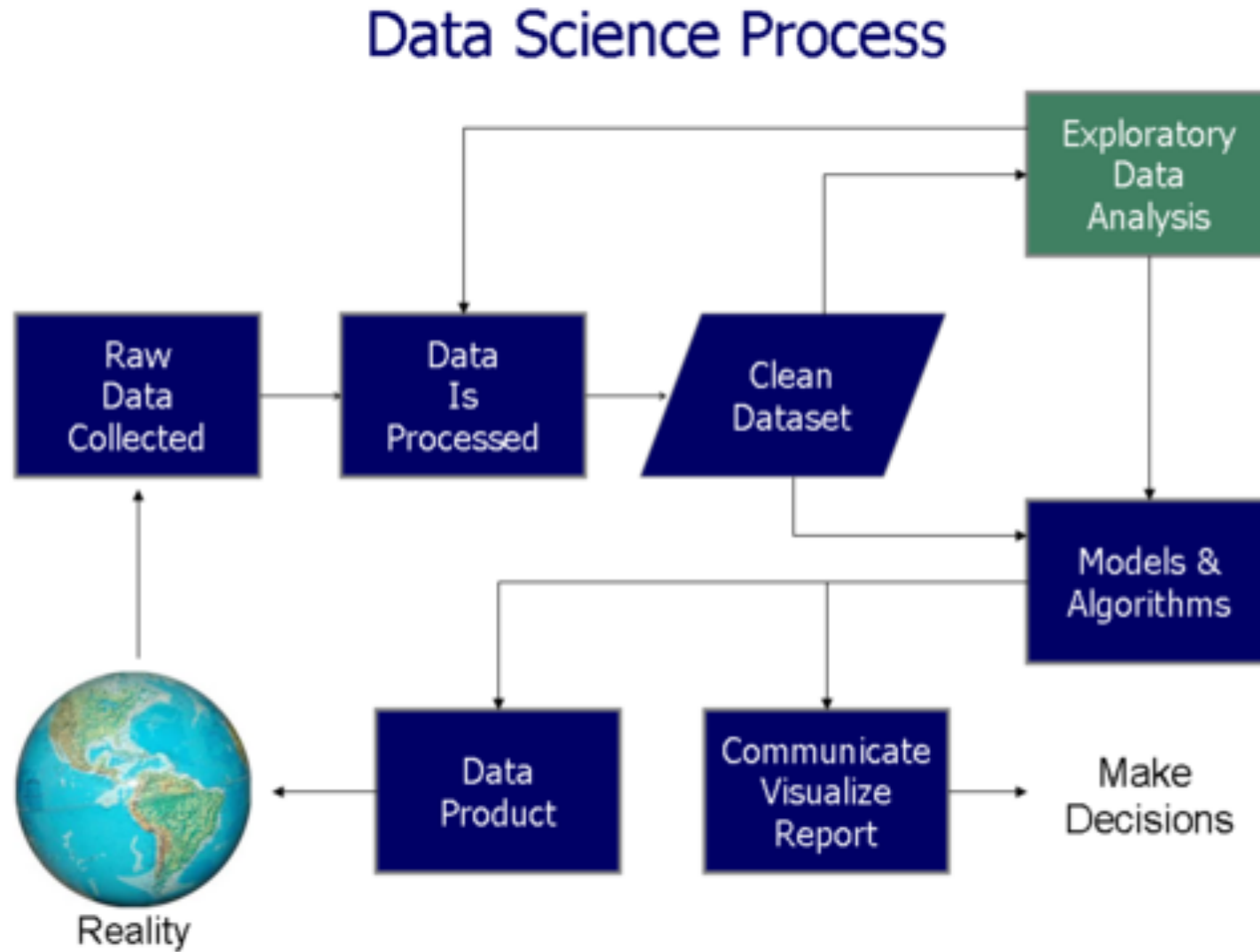
Start with a graph or graphs



Add numerical summaries

```
1-Var Stats
x̄=27
Σx=648
Σx²=17992
Sx=4.643836495
σx=4.546060566
n=24
```

Data Analysis



EDA 過程：

- 蒐集並清理資料
- 以視覺化的方式快速篩選特徵
- 得到據以建模的一套特徵值
- 建模完成
- 模型預測的和實際情況比較
- 持續優化

學習目標：

- 認識視覺化模組

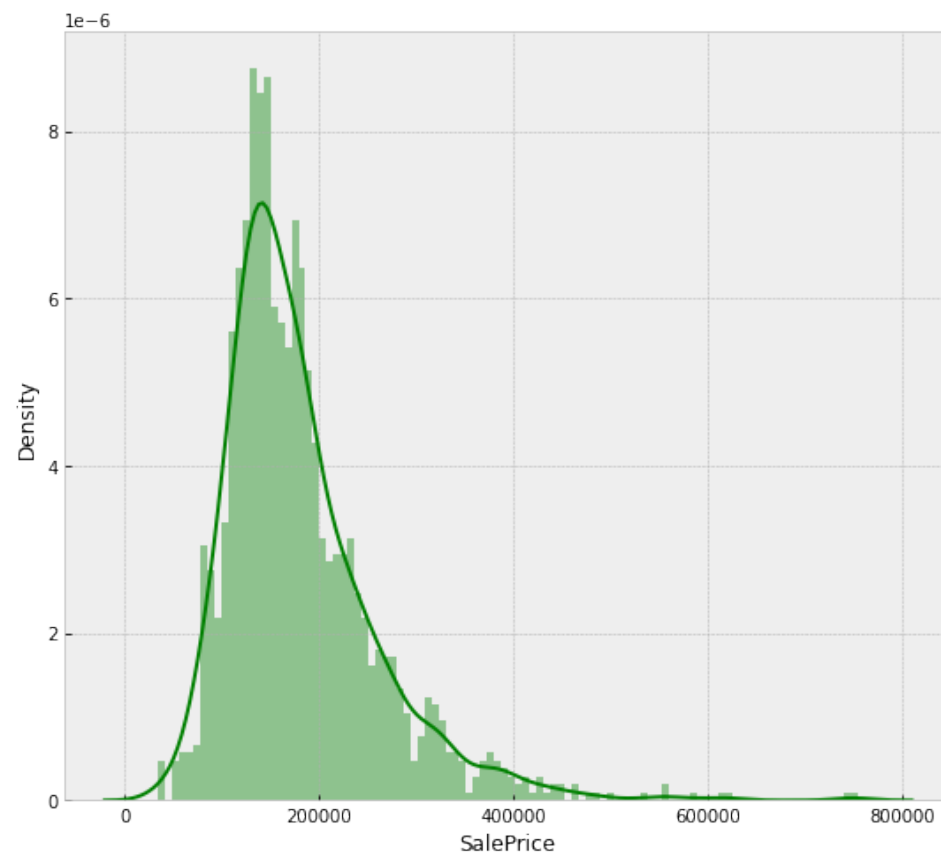
Module 5.

資料探索與視覺化

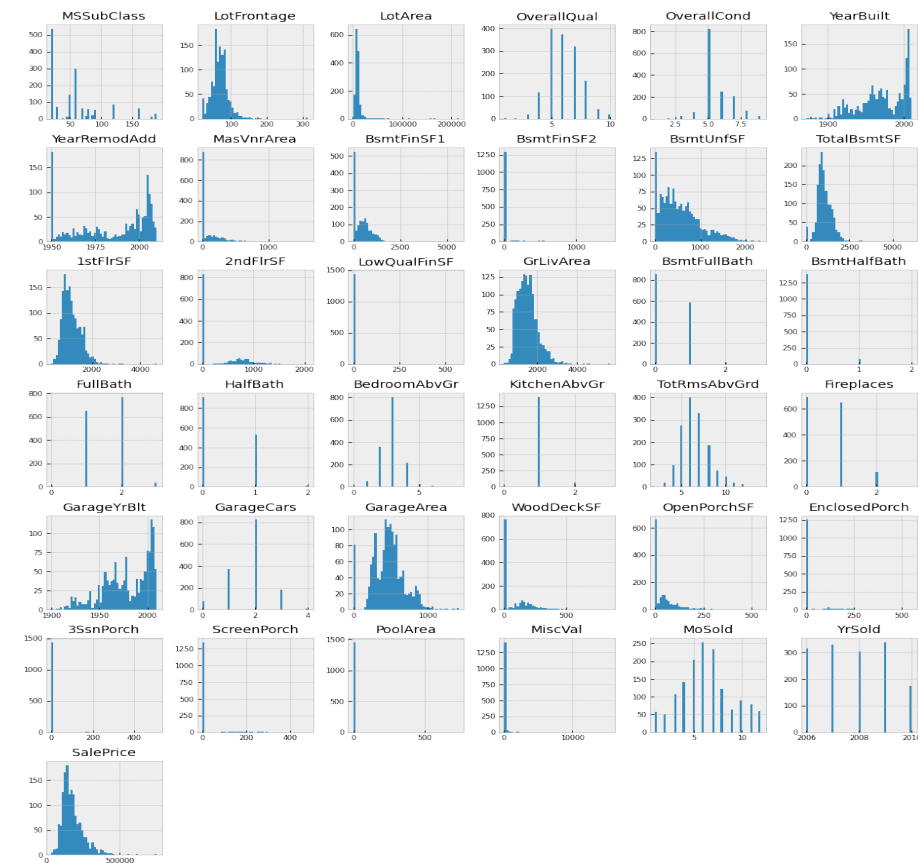
繪製圖表

以圖表視覺化統計數字以求一目瞭然，繼續探索。

1. 房價分布圖

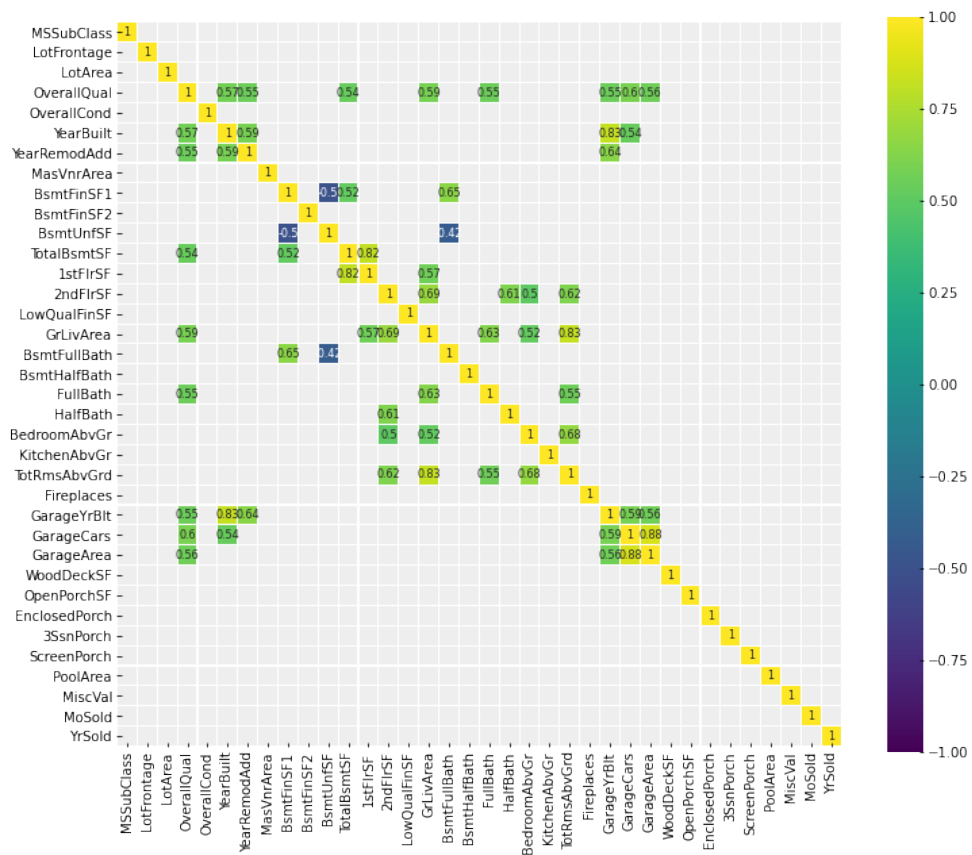


2. 各特徵的直方圖 (Histogram)

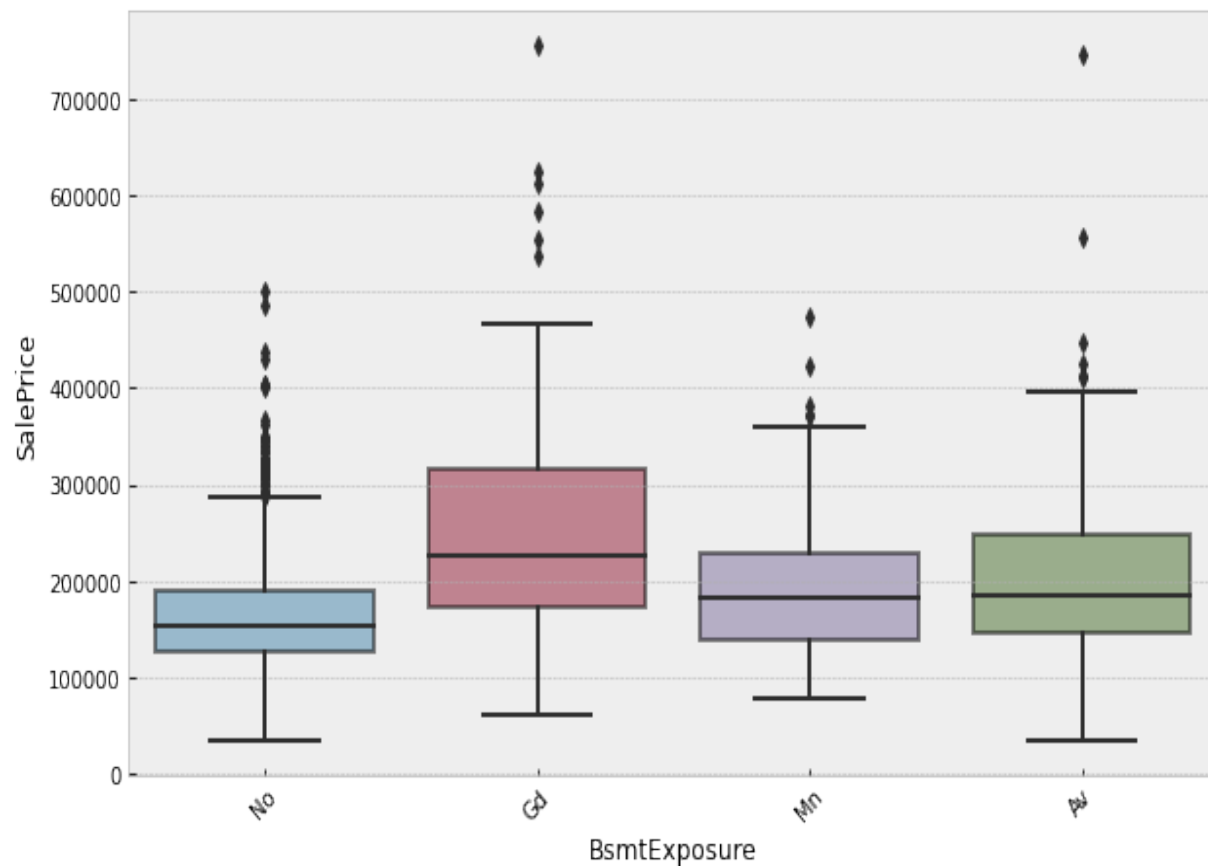


以圖表視覺化統計數字以求一目瞭然，繼續探索。

3. 房價與各特徵的關聯圖矩陣



4. 類別型特徵和房價間之盒鬚圖 (boxplot)



學習目標：

- 學會視覺化實例

Module 5.

資料探索與視覺化

資料探索實作

detailed-exploratory-data-analysis-with-python.ipynb

Data : train.csv

學習目標：

- 認識何謂關聯分析

Module 6.

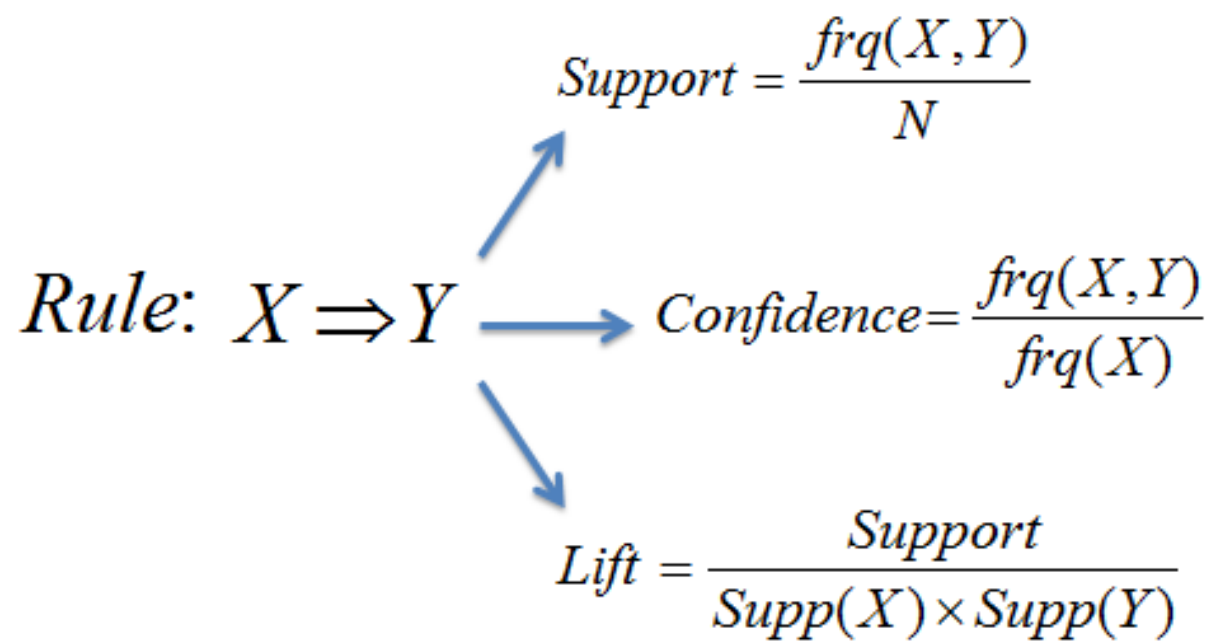
關聯分析

關聯分析簡介

關聯（規則）分析（Association Rules Analysis）

- 在交易數據、關係數據或其他信息載體中，查找存在於項目集合或對象集合之間的頻繁模式、關聯、相關性或因果結構。
- 發現存在於大量數據集中的關聯性或相關性，描述一個事物中**某些屬性同時出現**的規律和模式。
- 從大量數據中發現項集之間有趣的關聯和相關聯繫。
- 可從資料庫中關聯分析出形如“由於某些事件的發生而引起另外一些事件的發生”之類的規則。

關連分析三個重要的術語：何謂 Support, Confidence and Lift?



Rule	Support	Confidence	Lift
$A \Rightarrow D$	2/5	2/3	10/9
$C \Rightarrow A$	2/5	2/4	5/6
$A \Rightarrow C$	2/5	2/3	5/6
$B \& C \Rightarrow D$	1/5	1/3	5/9

學習目標：

- 熟悉 Apriori 演算法
- 熟悉 FP-growth 演算法

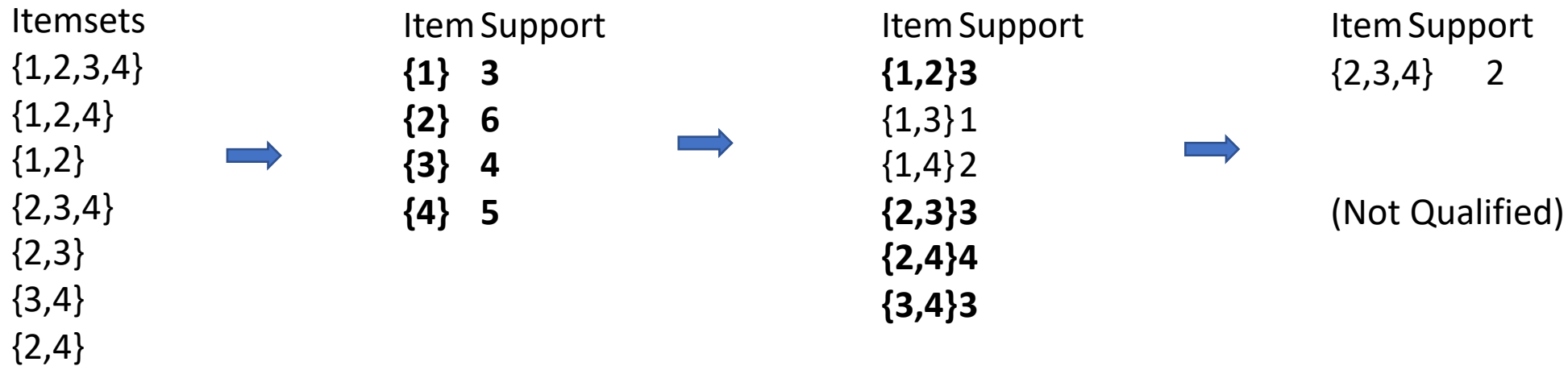
Module 6. 關聯分析

Apriori, FP-growth 演算法

Apriori (先驗) 演算法

- 挖掘產生布爾關聯規則的基本演算法，也是最著名的關聯規則挖掘演算法之一。
- 根據有關頻繁項集特性的先驗知識而命名的。
- 由下而上，使用稱作逐層搜索的迭代方法， k — 項集用於探索 $(k+1)$ — 項集。
- 找出頻繁 1 — 項集的集合，記做 $L1$ ， $L1$ 用於找出頻繁 2 — 項集的集合 $L2$ ，再用於找出 $L3$ ，如此下去，直到不能找到頻繁 k — 項集。
- 利用重要性質：一個頻繁項集的任一子集也應該是頻繁項集，可提高按層次搜索並產生相應頻繁項集的處理效率。

Apriori 的算法，假設 min. support threshold = 3

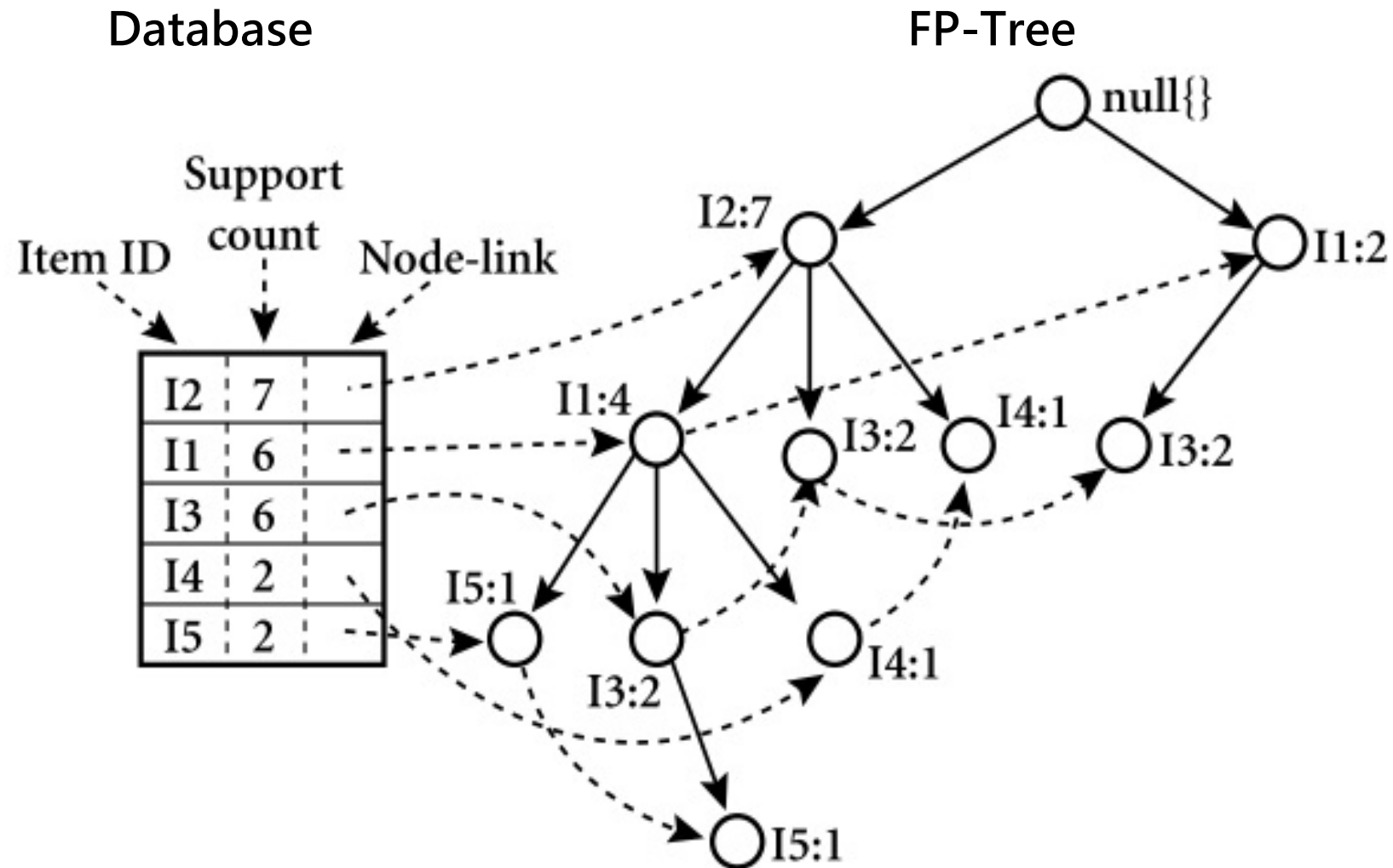


- Apriori 演算法缺點：
 1. 佔用資料庫很多空間
 2. 運算成本很高
- FP growth 演算法可改進以上缺點

FP-growth 演算法

- 通過兩次掃描事務資料庫 ➤ 按其支持度降序壓縮存儲到 FP—tree 中。
- 不需再掃描事務資料庫，可直接在 FP-Tree 中進行查找 ➤ 通過遞歸調用 FP-growth 的方法來直接產生頻繁模式。
- 過程中不需產生候選模式。
- 在執行效率上明顯好於 Apriori 演算法。

FP-growth 演算法資料結構



學習目標：

- 學會關聯分析實作

Module 6. 關聯分析

實作 - 購物籃分析

- [DEMO : https://pbpython.com/market-basket-analysis.html](https://pbpython.com/market-basket-analysis.html)
- <https://medium.com/@rodrigodutcosky/market-basket-analysis-with-apriori-algorithm-practical-implementation-ccadd7fe3dbd>
- <https://www.kdnuggets.com/2019/12/market-basket-analysis.html>

學習目標：

- 認識 Python ND Array

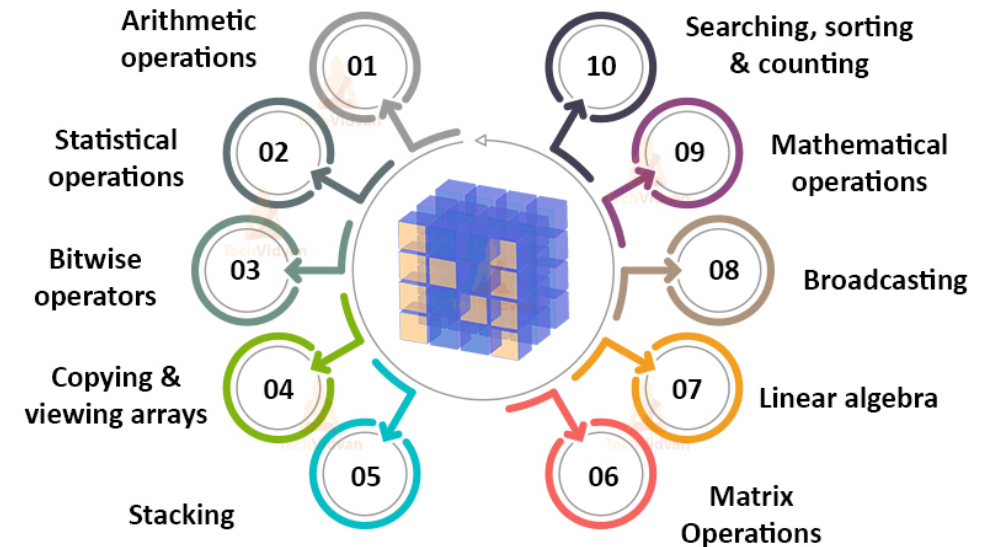
Module 7. Numpy套件

ND array

NumPy 程式庫提供：

- 多維物件（也就是 Nddarray）
- 各種衍生物件（such as masked arrays and matrices）
- 在矩陣上運作的各種常用程式

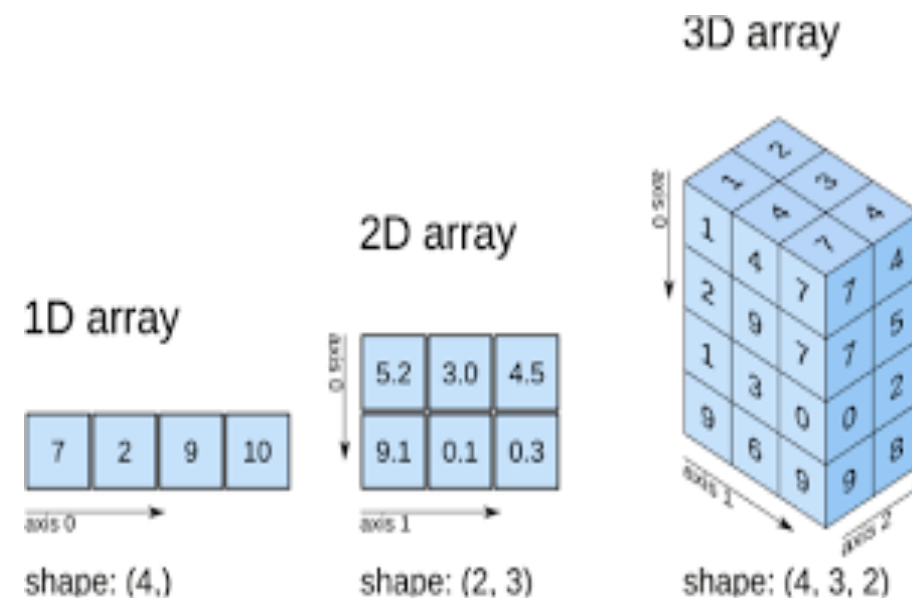
Uses of NumPy



pic from techvidvan.com

甚麼是 Nddarray :

- Nddarray 就是一個 (往往是固定大小) , 資料類別 (dtype) 相同的元素所構成的 N 維容器 (N-Dimensional Container)
- 此容器 / 矩陣中的維度和各維度大小我稱為形狀 (Shape)
- Tensor 基本上就是 Nddarray 另一種說法



pic from: itread01.com

程式舉例

說明	程式碼	輸出
#由 4-byte 整數所構成 2 維 2 x 3 矩陣	<pre>x = np.array([[1, 2, 3], [4, 5, 6]], np.int32) type(x)</pre>	<pre><class 'numpy.ndarray' ></pre>
	<pre>x.shape</pre>	<pre>(2, 3)</pre>
	<pre>x.dtype</pre>	<pre>dtype('int32')</pre>
	<pre>x[1, 2]</pre>	<pre>6</pre>
#只取中間那行	<pre>y = x[:,1] y</pre>	<pre>array([2, 5])</pre>
# 這也改變x相對元素	<pre>y[0] = 9 y</pre>	<pre>array([9, 5])</pre>
	<pre>x</pre>	<pre>array([[1, 9, 3], [4, 5, 6]])</pre>

學習目標：

- 了解 Numpy 基本運算

Module 7. Numpy套件

Numpy Basic Operation

Ndarray 基本算術：

Add, Subtract, Multiply, Divide ND Array

Ndarray 基本數學函數：

Exponent, Square Root, Sine / Cosine, Logarithm, Round

Ndarray 矩陣運算：

Dot product, Transpose, Inverse...

Numpy Basic Operation

說明	程式碼	輸出
	<pre>A = np.array([[3,2],[0,1]]) B = np.array([[3,1],[2,1]]) A+B</pre>	<pre>array([[6, 3], [2, 2]])</pre>
# element wise multiplication	<pre>A*B</pre>	<pre>array([[9, 2], [0, 1]])</pre>
# matrix multiplication	<pre>A @ B</pre>	<pre>array([[13, 5], [2, 1]])</pre>
# dot product	<pre>A.dot(B)</pre>	<pre>array([[13, 5], [2, 1]])</pre>
# reshape	<pre>A = np.arange(9).reshape(3,3) A</pre>	<pre>array([[0, 1, 2], [3, 4, 5], [6, 7, 8]])</pre>

學習目標：

- 了解 Broadcasting 運作

Module 7. Numpy 套件

Broadcasting

Numpy broadcasting rule 使得

```
>>> from numpy import array
>>> a = array([1.0, 2.0, 3.0])
>>> b = array([2.0, 2.0, 2.0])
>>> a * b
array([ 2.,  4.,  6.]
```

等同於

```
>>> from numpy import array
>>> a = array([1.0,2.0,3.0])
>>> b = 2.0
>>> a * b
array([ 2.,  4.,  6.]
```

也就是 b 由純量被視為等同向量

Broadcasting

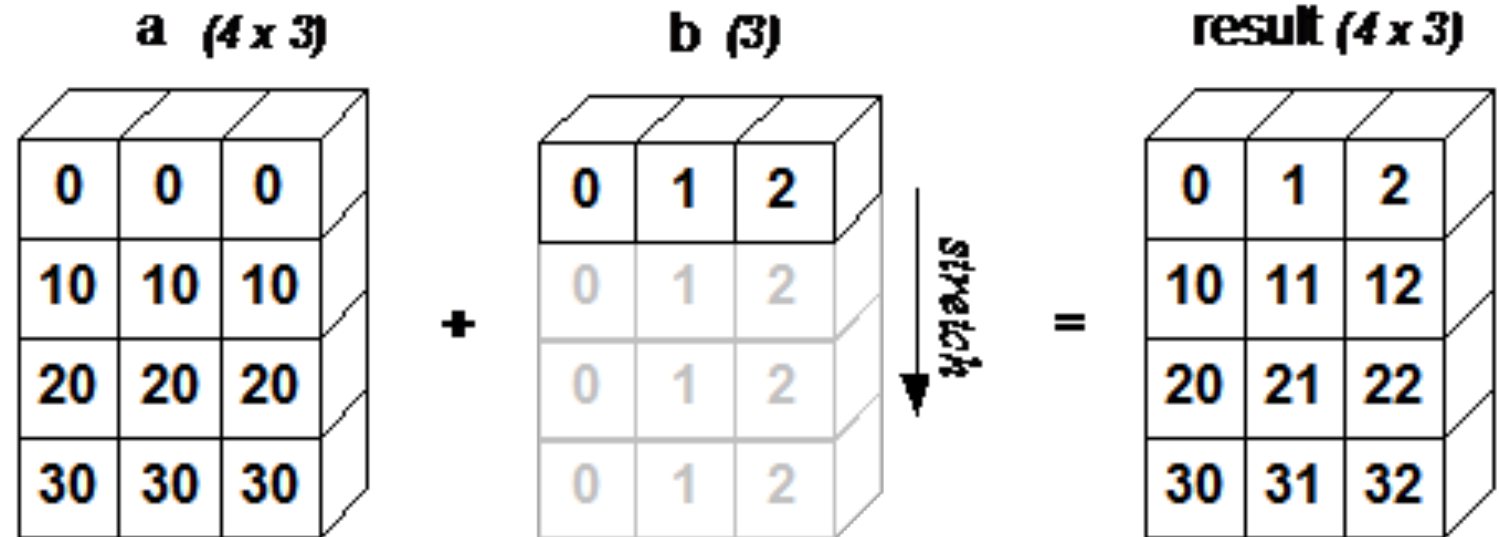
```
>>> from numpy import array
```

```
>>> a = array([[ 0.0,  0.0,  0.0],
...           [10.0, 10.0, 10.0],
...           [20.0, 20.0, 20.0],
...           [30.0, 30.0, 30.0]])
```

```
>>> b = array([1.0, 2.0, 3.0])
```

```
>>> a + b
```

```
array([[ 1.,  2.,  3.],
       [11., 12., 13.],
       [21., 22., 23.],
       [31., 32., 33.]])
```



學習目標：

- 了解 NDArray Index

Module 8. Nddarray 操作

index 取值

Ndarray 的特色：以 Index 取值

程式碼	輸出
<code>x = np.arange(10)</code> <code>x</code>	<code>array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])</code>
<code>x[2]</code>	<code>2</code>
<code>x[-2]</code>	<code>8</code>
<code>x.shape = (2,5)</code> <code>x[1,3]</code>	<code>8</code>
<code>x[1,-1]</code>	<code>9</code>
<code>x = np.arange(10,1,-1)</code> <code>x</code>	<code>array([10, 9, 8, 7, 6, 5, 4, 3, 2])</code>
<code>x[np.array([3, 3, 1, 8])]</code>	<code>array([7, 7, 9, 2])</code>

程式碼	輸出
<code>x = np.arange(10)</code> <code>x[2:5]</code>	<code>array([2, 3, 4])</code>
<code>x[:-7]</code>	<code>array([0, 1, 2])</code>
<code>x[1:7:2]</code>	<code>array([1, 3, 5])</code>
<code>y =</code> <code>np.arange(35).reshape(5,7)</code> <code>y[1:5:2,::3]</code>	<code>array([[7, 10, 13],</code> <code> [21, 24, 27]])</code>
<code>x=np.arange(10,1,-1)</code> <code>x[np.array([3,3,-3,8])]</code>	<code>array([7, 7, 4, 2])</code>
<code>x[np.array([[1,1],[2,3]])]</code>	<code>array([[9, 9],</code> <code> [8, 7]])</code>

學習目標：

- 認識 Mask Array
- 認識 Fancy Indexing

Module 8. Nddarray 操作

Mask & Fancy Indexing

甚麼是 Masked Array ?

- 由 Boolean (1/0) 值構成的 Array，像是 `mask=[0, 0, 0, 1, 0]`，被 Mask 掉的就不計入運算。

```
import numpy as np
import numpy.ma as ma
x = np.array([1, 2, 3, -1, 5])
```

```
mx = ma.masked_array(x, mask=[0, 0, 0, 1, 0])
```

```
mx.mean()
2.75
```

(因為 $(1+2+3+5)/4 = 2.75$)

甚麼是 Fancy Indexing?

- Fancy Index 就是我們不用數值而用矩陣當作 Index，好處是我們能很快得到或改變矩陣部分的值甚至 shape。

```
import numpy as np
```

```
rand = np.random.RandomState(42)  
x = rand.randint(100, size=10)  
print(x)  
[51 92 14 71 60 20 82 86 74 74]
```

```
[x[3], x[7], x[2]]  
[71, 86, 14]
```

```
ind = [3, 7, 4]  
x[ind]  
array([71, 86, 60])
```

```
ind = np.array([[3, 7],  
                [4, 5]])  
x[ind]  
  
array([[71, 86],  
       [60, 20]])
```

學習目標：

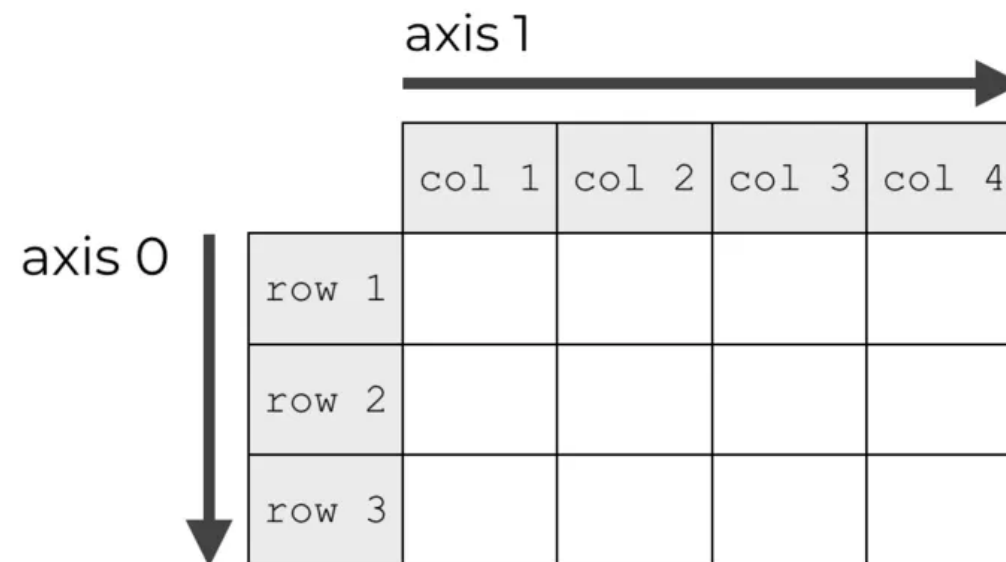
- 了解 Axis 運算原理

Module 8. Nddarray 操作

Axis

NumPy axes (axis 的複數)

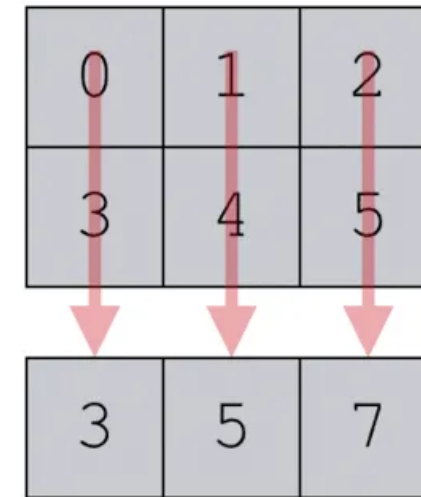
- Nddarray 軸 0 是垂直方向沿著 rows
- 軸 1 則是水平方向沿 columns




```
np_array_2d = np.arange(0, 6).reshape([2,3])  
print(np_array_2d)  
[[0 1 2]  
 [3 4 5]]
```

```
np.sum(np_array_2d, axis = 0)  
array([3, 5, 7])
```

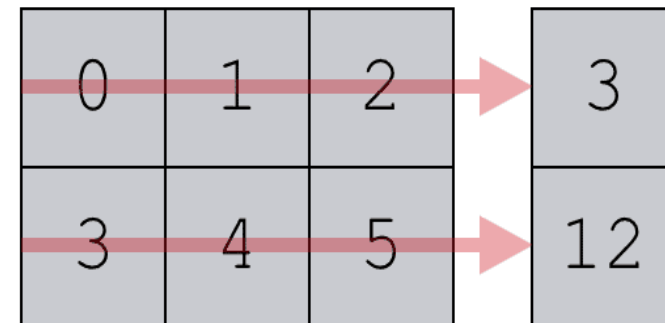
WHEN WE SET `axis = 0`, `np.sum()` COLLAPSES THE ROWS AND CALCULATES THE SUM



```
np_array_2d = np.arange(0, 6).reshape([2,3])  
print(np_array_2d)  
[[0 1 2]  
 [3 4 5]]
```

```
np.sum(np_array_2d, axis = 1)  
array([3, 12])
```

WHEN WE SET `axis = 1`, `np.sum()` COLLAPSES THE COLUMNS AND CALCULATES THE SUM



學習目標：

- 認識 Numpy 統計運算元

Module 8. Nddarray 操作

Statistical Operators

Ndarray 有自帶的統計運算子，也有套件提供較複雜的統計運算函數

- 順序統計 (Order Statistics) : amin, amax, ptp, percentile, quantile.....
- 均值與變異數 (Averages and Variances) : median, average, mean, std, var.....
- 關聯 (Correlating) 函數 : corrcoef, correlate, cov.....
- 直方圖 (Histogram) : histogram, histogram2d, bincount....

展示 : NDarrayIntro.ipynb

學習目標：

- 了解數值矩陣合併之連接

Module 9.

Ndarray 合併與轉換

Concatenate

陣列連接：

陣列合併與重塑或轉換的函數之一，可以是橫向或縱向，留意 Axis 的數值。

`numpy.concatenate((a1, a2, ...), axis=0, out=None)`

- `a1, a2`：要連接的陣列
- `axis`：0 是縱向連接，1 是橫向，default = 縱向
- `out`：輸出陣列

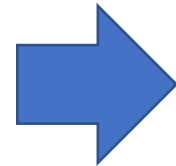
例題：

```
array2D_1 = np.arange(9).reshape((3,3))  
array2D_1
```

```
array([[0, 1, 2],  
       [3, 4, 5],  
       [6, 7, 8]])
```

```
array2D_2 =  
np.arange(10,19).reshape(3,3)  
array2D_2
```

```
array([[10, 11, 12],  
       [13, 14, 15],  
       [16, 17, 18]])
```



```
# concatenate 2 numpy arrays: row-wise  
np.concatenate((array2D_1, array2D_2))
```

```
array([[ 0,  1,  2],  
       [ 3,  4,  5],  
       [ 6,  7,  8],  
       [10, 11, 12],  
       [13, 14, 15],  
       [16, 17, 18]])
```

```
# concatenate 2 numpy arrays: column-wise  
np.concatenate((array2D_1,array2D_2),axis=1)
```

```
array([[ 0,  1,  2, 10, 11, 12],  
       [ 3,  4,  5, 13, 14, 15],  
       [ 6,  7,  8, 16, 17, 18]])
```

學習目標：

- 數值矩陣合併之堆疊

Module 9. Nddarray 合併與轉換

Stack

陣列疊加（stack）：

在新軸上**增加**更多的陣列序列（sequence），可以是橫向或縱向。

`numpy.stack(arrays, axis=0, out=None)`

```
arrays = [np.random.randn(3, 4) for _ in  
range(10)]  
np.stack(arrays, axis=0).shape  
(10, 3, 4)
```

```
np.stack(arrays, axis=1).shape  
(3, 10, 4)
```

```
np.stack(arrays, axis=2).shape  
(3, 4, 10)
```

陣列疊加 (stack) :

在新軸上增加更多的陣列序列 (sequence) , 可以是橫向或縱向。

```
numpy.stack(arrays, axis=0, out=None)
```

```
a = np.array([1, 2, 3])  
b = np.array([2, 3, 4])  
np.stack((a, b))
```

```
array([[1, 2, 3],  
       [2, 3, 4]])
```

```
np.stack((a, b), axis=-1)  
array([[1, 2],  
       [2, 3],  
       [3, 4]])
```

學習目標：

- 轉換至其他資料形式

Module 9.

Ndarray

合併與轉換

Ndarray to other Data Types

陣列數值種類轉換

`ndarray.astype(dtype, order='K', casting='unsafe', subok=True, copy=True)`

- `dtype` : 資料種類
- `order` : { 'C' , 'F' , 'A' , 'K' }, optional C / Fortran 語言順序
- `Casting` : { 'no' , 'equiv' , 'safe' , 'same_kind' , 'unsafe' }, optional
- `Returns` : ndarray

舉例 :

```
x = np.array([1, 2, 2.5])
```

x

```
array([1., 2., 2.5])
```

```
x.astype(int)
```

```
array([1, 2, 2])
```

學習目標：

- 認識 DF 中 Axis 扮演的角色

Module 10. Pandas 物件運算

Axis in dataframe

DataFrame 的軸 (Axis)

- DataFrame 如 SQL table 有行和列，行由 Series 所構成。
- Axis = 0 指的是縱 (列) 向；Axis = 1 指的是橫 (行) 向。

```
import pandas as pd
srs_a = pd.Series([10,20,30,80,90])
srs_b = pd.Series([ 'red' , 'green' , 'blue' , 'white' , 'black' ])
df = pd.DataFrame({ 'a' : srs_a, 'b' : srs_b })
df
```

		Axis 1	
		a	b
Axis 0	0	10	red
	1	20	green
	2	30	blue
	3	80	white
	4	90	black

```
import pandas as pd
srs_a = pd.Series([1,3,6,8,9])
srs_b = pd.Series(['red', 'green', 'blue', 'white', 'black'])
df = pd.DataFrame({'a': srs_a, 'b': srs_b})
df.loc[2, 'b']
'blue'
df.loc[3, 'a']
8
```

		Axis 1	
		a	b
Axis 0	0	1	red
	1	3	green
	2	6	blue
	3	8	white
	4	9	black

```
import pandas as pd
srs_a = pd.Series([10,30,60,80,90])
srs_b = pd.Series([22, 44, 55, 77, 101])
df = pd.DataFrame({'a': srs_a, 'b':
srs_b})
df
   a  b
0 10 22
1 30 44
2 60 55
3 80 77
4 90 101
#縱向加總
df.sum(axis=0)
a    270
b    299
dtype: int64
```

```
#橫向加總
df.sum(axis=1)
0    32
1    74
2   115
3   157
4   191
dtype: int64
```

- “axis 0” 有兩個別名: ‘index’ , ‘rows’
- “axis 1” 只有一個: ‘columns’

學習目標：

- 認識 Pandas 資料種類

Module 10. Pandas 物件運算

Pandas data types

Pandas data types

Pandas dtype mapping			
Pandas dtype	Python type	NumPy type	Usage
object	str or mixed	string_, unicode_, mixed types	Text or mixed numeric and non-numeric values
int64	int	int_, int8, int16, int32, int64, uint8, uint16, uint32, uint64	Integer numbers
float64	float	float_, float16, float32, float64	Floating point numbers
bool	bool	bool_	True/False values
datetime64	NA	datetime64[ns]	Date and time values
timedelta[ns]	NA	NA	Differences between two datetimes
category	NA	NA	Finite list of text values

Pandas 三種改變 data types 的基本選擇：

- Use `astype()`
- 以客製函數改變
- 用 Pandas 函數如 `to_numeric()`，`to_datetime()`

舉例：

```
df_2 = pd.read_csv("sales_data_types.csv",  
                  dtype={'Customer Number': 'int'},  
                  converters={'2016': convert_currency,  
                              '2017': convert_currency,  
                              'Percent Growth': convert_percent,  
                              'Jan Units': lambda x: pd.to_numeric(x, errors='coerce'),  
                              'Active': lambda x: np.where(x == "Y", True, False)  
                  })
```

`df_2.dtypes`

學習目標：

- 認識 Pandas 遺失值
- 認識 Pandas 重複值

Module 10. Pandas物件運算

Handle Missing and Duplicated data

Pandas 處理遺失值的方法：

(請參考模組：資料清洗與轉換 (I)：遺失值偵測與填補)

- 將遺失值整列除去
- 將遺失值整行除去
- 以某些值 (前值，後值，均值) 填補遺失值
- 不處理

偵測遺失值：`df.isnull()`

填補遺失值：`df.fillna()`

除去遺失值：`df.dropna()`

- Pandas 之外，Numpy 本身也有偵測遺失值，Mask 遺失值之運算

Numpy 偵測遺失值

```
np.isnan(np.nan)
```

```
True
```

```
np.isnan(np.inf)
```

```
False
```

```
np.isnan([np.log(-1.),1.,np.log(0)])
```

```
array([ True, False, False])
```

Numpy Mask 遺失值

sample input ndarray:

```
x = np.array([1.0, 2.5, np.nan, 1.3, np.inf, 7.2])
```

```
x
```

```
[1.  2.5 nan 1.3 inf 7.2]
```

```
xm = np.ma.masked_invalid(x)
```

```
xm
```

```
[1.0 2.5 -- 1.3 -- 7.2]
```

Masked 值之運算

```
x = np.ma.array([1, 2, 3], mask=[False, False, True])
```

```
y = np.ma.array([1, 0, 1])
```

```
print(x * y)
```

```
[1 0 --]
```

```
print(x/y)
```

```
[1.0 -- --]
```

Handle Missing and Duplicated data

- Pandas 處理重複值的方法：
偵測重複值

假設已輸入 data DF 如中圖
以 first name 排序

偵測 first name 重複
bool_series = data["First Name"].duplicated()

展現數據
data.head()

展現重複數據
data[bool_series]

重複資料

	First Name	Gender	Start Date	Last Login Time	Salary	Bonus %	Senior Management	Team
327	Aaron	Male	1/29/1994	6:48 PM	58755	5.097	True	Marketing
440	Aaron	Male	7/22/1990	2:53 PM	52119	11.343	True	Client Services
937	Aaron	NaN	1/22/1986	7:39 PM	63126	18.424	False	Client Services
141	Adam	Male	12/24/1990	8:57 PM	110194	14.727	True	Product
302	Adam	Male	7/5/2007	11:59 AM	71276	5.027	True	Human Resources
538	Adam	Male	10/8/2010	9:53 PM	45181	3.491	False	Human Resources
53	Alan	NaN	3/3/2014	1:28 PM	40341	17.578	True	Finance
610	Alan	Male	2/17/2012	12:26 AM	41453	10.084	False	Product
458	Albert	Male	9/30/2007	5:34 PM	102626	15.843	False	Finance
477	Albert	Male	8/14/2008	5:43 PM	137840	9.705	False	Business Development
680	Albert	Male	6/17/1992	3:25 AM	86818	14.301	True	Engineering
999	Albert	Male	5/15/2012	6:24 PM	129949	10.169	True	Sales
959	Albert	Male	9/19/1992	2:35 AM	45094	5.850	True	Business Development
970	Alice	Female	9/3/1988	8:54 PM	63571	15.397	True	Product
765	Alice	NaN	4/23/1995	6:35 AM	148339	11.479	True	Finance

Bool_series

101 False
327 True
440 True
937 True
137 False
...
902 True
925 True
946 True
947 True
951 True

- Pandas 處理重複值的方法：
除去重複值

```
# 假設已輸入 data DF 如中圖
# 以 first name 排序
```

```
# 偵測 first name 重複
bool_series = data["First
Name"].duplicated()
```

```
# bool series
bool_series
# 過濾掉非重複的
data = data[~bool_series]
# 展示非重複資料
# 右上圖
data.info()
# 右下圖
data
```

```
Data columns (total 8 columns):
First Name      9 non-null object
Gender          9 non-null object
Start Date      9 non-null object
Last Login Time 9 non-null object
Salary          9 non-null int64
Bonus %         9 non-null float64
Senior Management 9 non-null object
Team            9 non-null object
dtypes: float64(1), int64(1), object(6)
memory usage: 432.0+ bytes
```

]:

	First Name	Gender	Start Date	Last Login Time	Salary	Bonus %	Senior Management	Team
8	Angela	Female	11/22/2005	6:29 AM	95570	18.523	True	Engineering
688	Brian	Male	4/7/2007	10:47 PM	93901	17.821	True	Legal
190	Carol	Female	3/19/1996	3:39 AM	57783	9.129	False	Finance
887	David	Male	12/5/2009	8:48 AM	92242	15.407	False	Legal
5	Dennis	Male	4/18/1987	1:35 AM	115163	10.125	False	Legal
495	Eugene	Male	5/24/1984	10:54 AM	81077	2.117	False	Sales
33	Jean	Female	12/18/1993	9:07 AM	119082	16.180	False	Business Development
832	Keith	Male	2/12/2003	3:02 PM	120672	19.467	False	Legal
291	Tammy	Female	11/11/1984	10:30 AM	132839	17.463	True	Client Services

學習目標：

- 了解 Matplotlib Figure 的用法

Module 11. Matplotlib 套件

figure

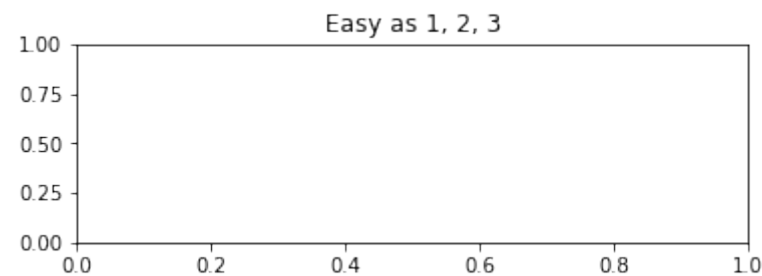
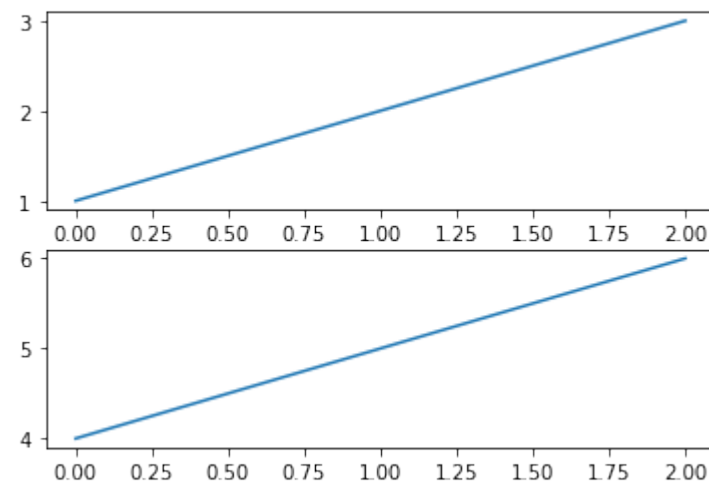
Matplotlib :

利用 Python 所實作的繪圖套件，其中包含兩個最重要的模組 — pylab 和 pyplot。

Pyplot 呼叫順序：

- 創造一個 figure
- 在 figure 中創造一個 plotting area
- plots 一些 lines 用 labels 裝飾 plot

展示：[Tibame_Matplotlib.ipynb](#)



學習目標：

- 了解 Subplot 功能

Module 11. Matplotlib 套件

Sub-plot

Subplot :

可界定一張圖上想要呈現幾個子圖。

參數的三個數字代表的是：(列數,行數,序號)

```
from matplotlib import pyplot as plt
```

```
#匯入 pyplot 套件
```

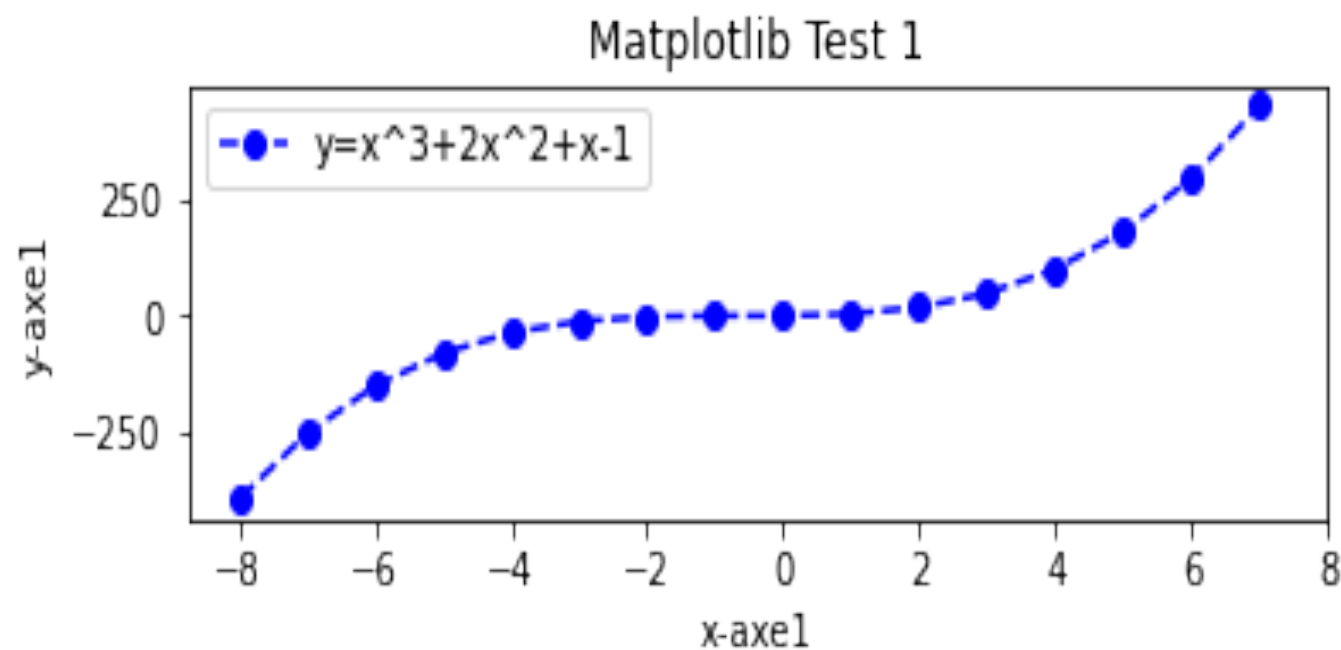
```
x = range(-8, 8)
```

```
y = [i**3 + 2*i**2 + i - 1 for i in x]
```

```
plt.subplot(211)
```

```
#.....
```

```
#.....
```



展示 : [Tibame_Matplotlib.ipynb](#)

學習目標：

- 學會繪製各種圖表

Module 11.

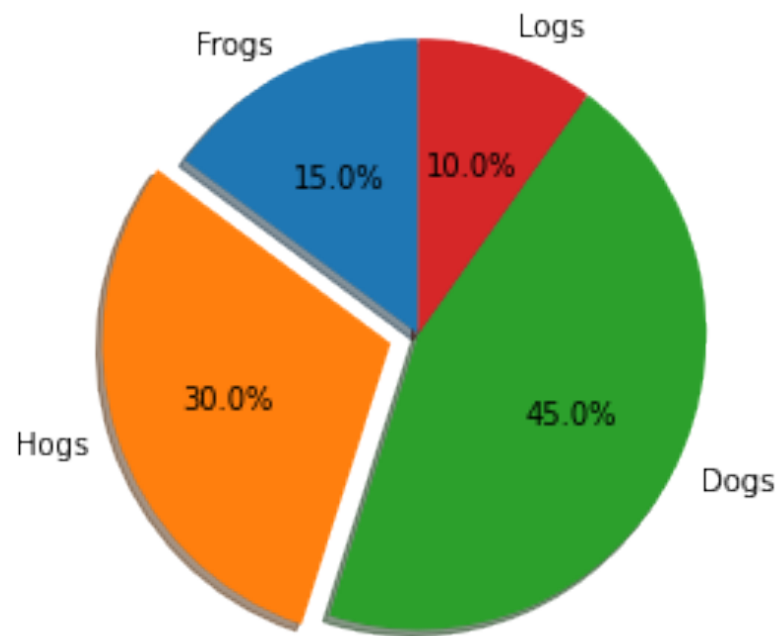
Matplotlib 套件

pie chart, scatter,
histogram, bar chart

PIE Chart的繪製：

繪製步驟：

- # 每一片都必須反時鐘依序繪出
- # 要有標籤
- # 大小比例
- # 哪片要突出
- # 在 subplot ax1 上畫 pie
- # 軸線比例一樣才會圓
- # 出現在畫布上



展示：[Tibame_Matplotlib.ipynb](#)

Scatter Chart 的繪製：

繪製步驟：

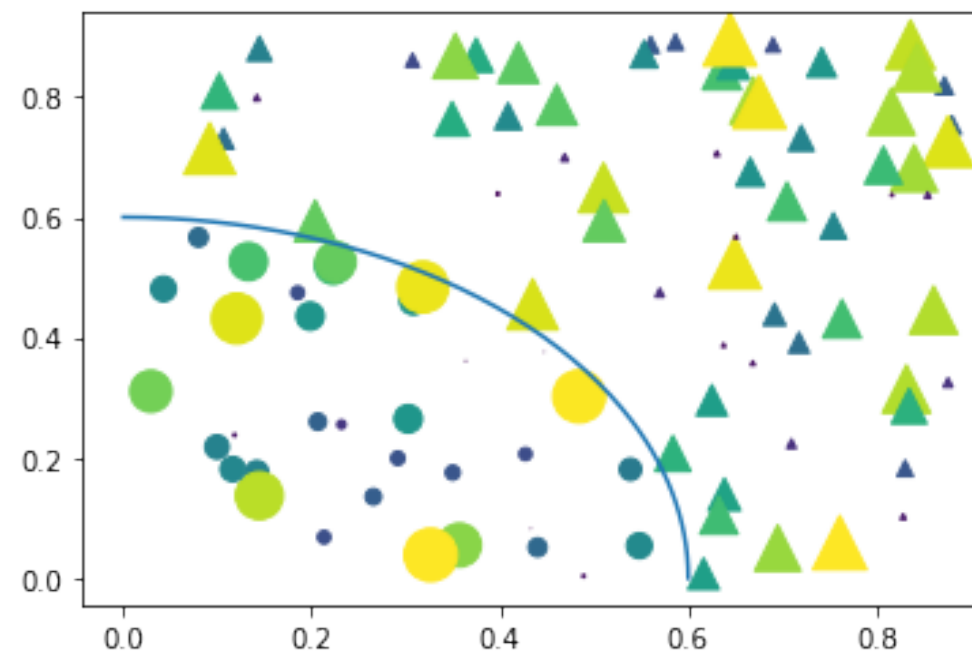
任意產生各 100 散點圖

固定隨機變數以便重繪時一樣

任意散點三角形落在半徑 0.6 之外

任意散點圓型落在半徑 0.6 之內

繪出分隔線



展示：[Tibame_Matplotlib.ipynb](#)

Histogram 的繪製：

繪製步驟：

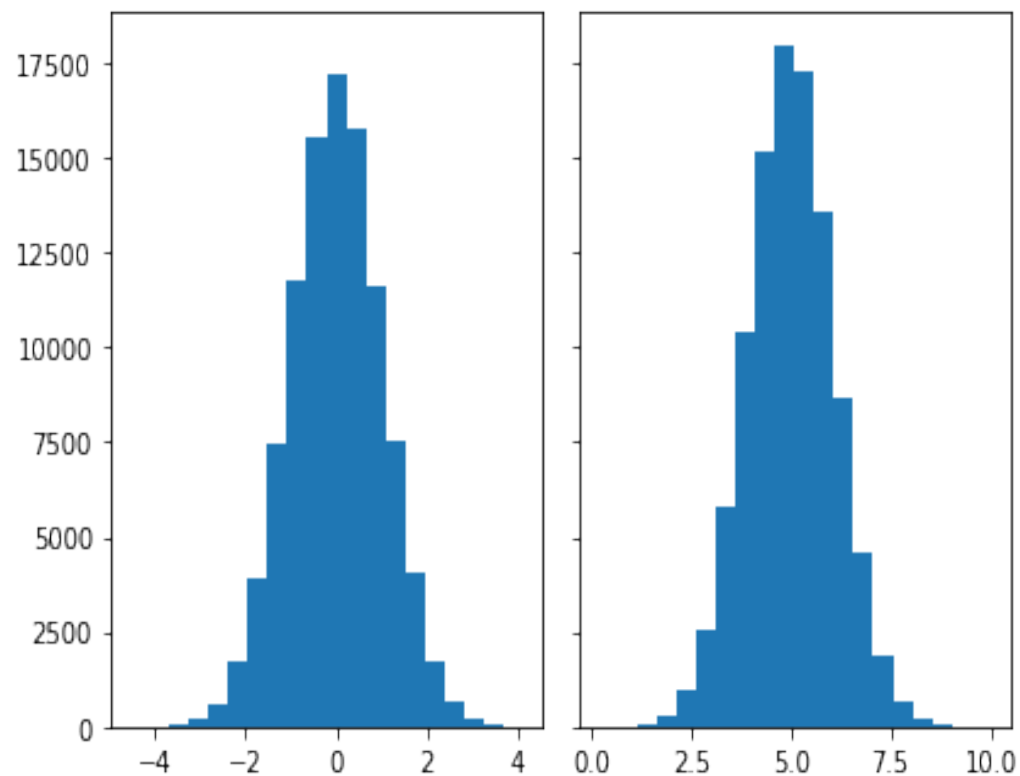
固定隨機變數以便重繪時一樣

產生中心點在 $x=0, y=5$ 的常態分佈

bin 的數目可自訂

繪製 x 的直方圖分佈

繪製 y 的直方圖分佈



展示：Tibame_Matplotlib.ipynb

Bar Chart的繪製：

繪製步驟：

N = 5 條 bars

男女平均值與標準差

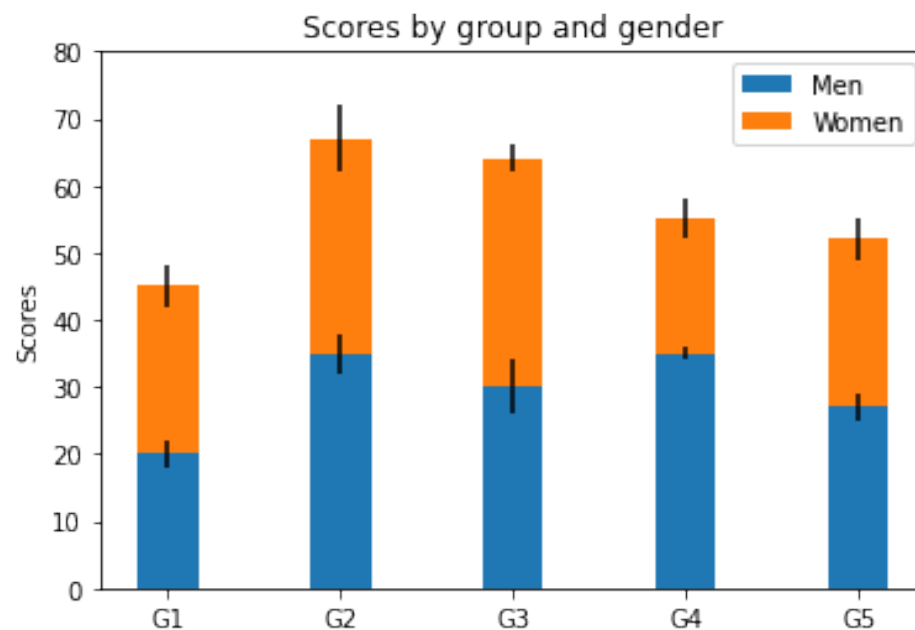
定好 5 bar 的 x 位置

bar 寬 width = 0.35

繪出 p1 男 bar

在男 bar 上繪女 bar

各種標註



Module 12.

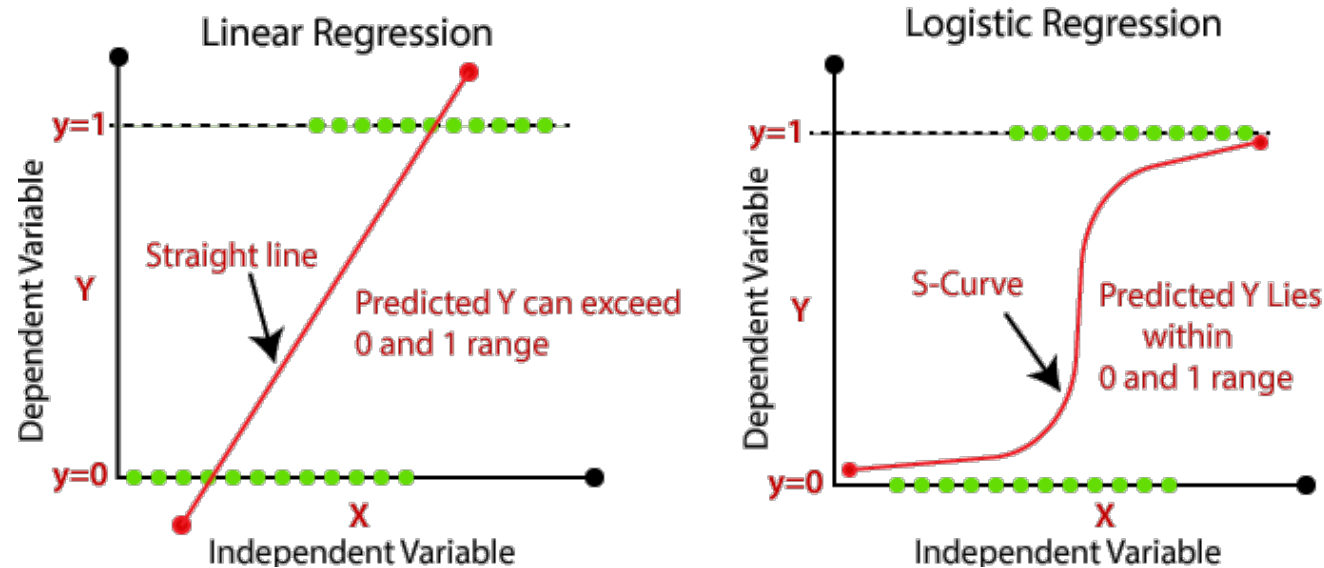
更多資料探勘 演算法簡介

學習目標：

- 線性回歸簡介
- 羅吉斯回歸簡介

線性與羅吉斯迴歸

- 線性迴歸 (Linear regression) :
是以一組獨立特徵去預測一個連續型的依變數，用來解決迴歸（預測）問題。
- 羅吉斯迴歸 (Logistic Regression) :
是用來預測類別型依變數，解決分類問題。



pic from: javapoint.com

線性迴歸 (Linear regression) 簡介

例題：

```
# 輸入套件
import numpy as np
from sklearn.linear_model import
LinearRegression
# x=原始資料
x = np.array([[1, 1], [1, 2], [2, 2], [2, 3]])
# y=目標值
#  $y = 1 * x_0 + 2 * x_1 + 3$ 
y = np.dot(X, np.array([1, 2])) + 3
reg = LinearRegression().fit(X, y)
```

```
# 精準度
reg.score(X, y)
1.0
# 斜率
reg.coef_
array([1., 2.])
# 截距
reg.intercept_
3.0000...
# 外插預測
reg.predict(np.array([[3, 5]]))
array([16.])
```

羅吉斯迴歸 (Logistic Regression) 簡介

例題：

([Iris 資料庫](#) 有 150 資料，X 有 4 個特徵值，y 是 0,1,2 三者之一)

輸入套件

```
from sklearn.datasets import
```

```
load_iris
```

```
from sklearn.linear_model import
```

```
LogisticRegression
```

載入資料

```
x, y = load_iris(return_X_y=True)
```

建立分類模型

```
clf =
```

```
LogisticRegression(random_state=0).
```

```
fit(X, y)
```

拿前兩筆預測

```
clf.predict(X[:2, :])
```

```
array([0, 0])
```

預測或然率

```
clf.predict_proba(X[:2, :])
```

```
array([[9.8...e-01, 1.8...e-02, 1.4...e-08],  
       [9.7...e-01, 2.8...e-02, ...e-08]])
```

精準度

```
clf.score(X, y)
```

```
0.97...
```

學習目標：

- 認識分類演算法簡介

Module 12.

更多資料探勘 演算法簡介

**分類：決策樹，
樸素貝葉斯與 SVM**

分類（**Classification**）：監督式學習，需要將資料標註類別，分為訓練集和測試集。

- **決策樹：**

定下一個最初的質點，從該點出發、根據最大**亂度下降**分叉，如此循環反覆。

- **樸素貝葉斯：**

若要推事件 A 發生的概率下 B 發生的概率，可以通過求事件 B 發生的概率下事件 A 發生的概率，再通過貝葉斯定理計算即可算出結果。

- **SVM：**

通過調節核函數參數的設置，將在二維上線性不可分的問題轉化為在多維上線性可分的問題，再尋找一個最優切割平面。

決策樹分類演算法簡介：

```
# 使用Iris Dataset 做分類
from sklearn.datasets import load_iris
from sklearn.model_selection import cross_val_score
from sklearn.tree import DecisionTreeClassifier
# 用Decision Tree 建模
clf = DecisionTreeClassifier(random_state=0)
# 載入資料
iris = load_iris()
# 精準度測試 cv 測 10次
cross_val_score(clf, iris.data, iris.target, cv=10)
... # doctest: +SKIP
...
array([ 1.    , 0.93..., 0.86..., 0.93..., 0.93...,
        0.93..., 0.93..., 1.    , 0.93..., 1.    ])
```



樸素貝葉斯演算法簡介：

```
import numpy as np
#載入X,Y
X = np.array([[ -1, -1], [-2, -1], [-3, -2], [1, 1], [2, 1],
               [3, 2]])
Y = np.array([1, 1, 1, 2, 2, 2])
from sklearn.naive_bayes import GaussianNB
#分類器的名稱叫做 GaussianNB()
clf = GaussianNB()
clf.fit(X, Y)
GaussianNB()
#以X=[-0.8,-1]預測Y=[1]
print(clf.predict([[ -0.8, -1]]))
[1]
```

Naive Bayes

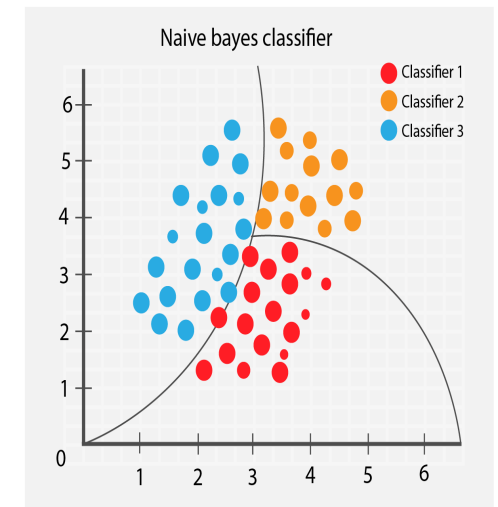


In machine learning, naive Bayes classifiers are a family of simple "probabilistic classifiers" based on applying Bayes' theorem with strong (naive) independence assumptions between the features.

$$P(A|B) = \frac{P(B|A) P(A)}{P(B)}$$

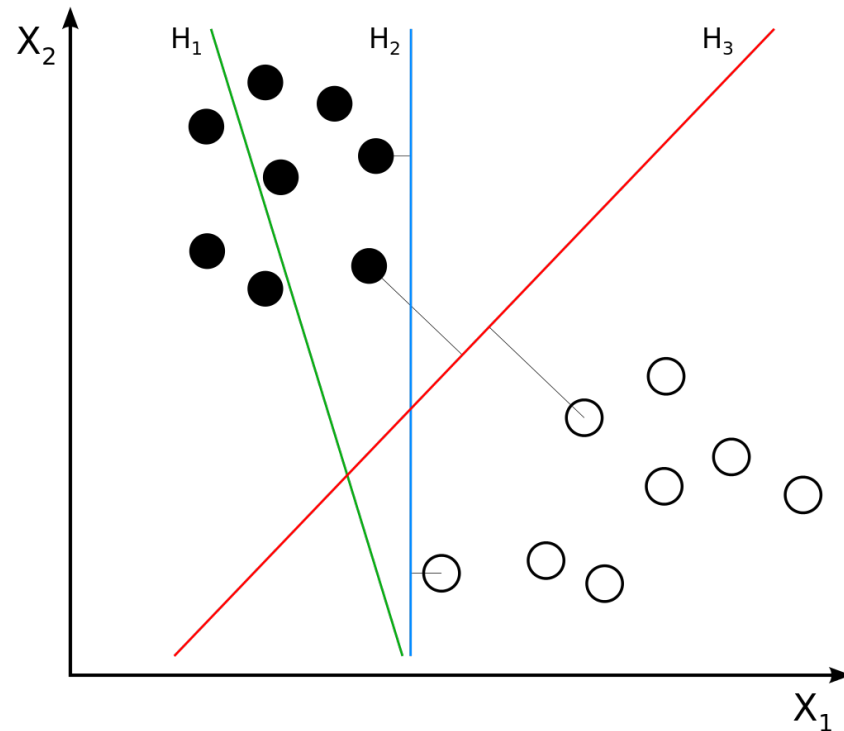
using Bayesian probability terminology, the above equation can be written as

$$\text{Posterior} = \frac{\text{prior} \times \text{likelihood}}{\text{evidence}}$$



pic from: towarddatascience.com

SVM 演算法簡介：



```
import numpy as np
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
# 載入資料X,y
X = np.array([[-1, -1], [-2, -1], [1, 1], [2, 1]])
Y = np.array([1, 1, 2, 2])
# SVM/SVC模組
from sklearn.svm import SVC
# SVM Classifier 的使用法
clf = make_pipeline(StandardScaler(),
                    SVC(gamma='auto'))
clf.fit(X, y)
Pipeline(steps=[('standardscaler', StandardScaler()),
                ('svc', SVC(gamma='auto'))])
```

學習目標：

- 認識分群演算法簡介

Module 12.

更多資料探勘 演算法簡介

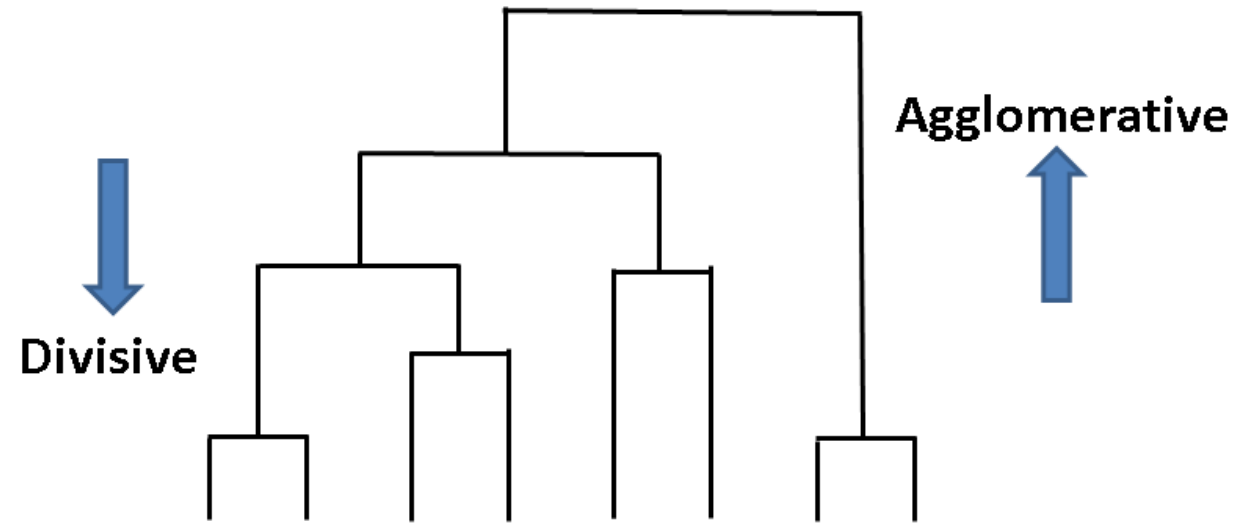
**分群：階層式分群，
K-means 與 DBSCAN**

分群 (Clustering)：一種非監督式學習，常用的分群演算法有階層式分群，K-means 與 DBSCAN。

- **階層式分群 (Hierarchical Clustering)**：
由許多單點逐漸合併成群，或由一個總群逐漸分裂成群。
- **K-means**：
先任意選定K個點當質心，並通過計算每一個樣本與質心之間的相似度，將樣本點歸到最相似的群中，重新計算每個群的質心，重複直到質心不再改變。
- **DBSCAN**：
如果點 p 的 Eps 鄰域包含的點多於 $MinPts$ 個，則建立一個以 p 為核心物件的簇(群)。並將 p 的 Eps 鄰域內所有點歸入 p 所在的簇。當沒有新的點新增到任何簇時，該過程結束。

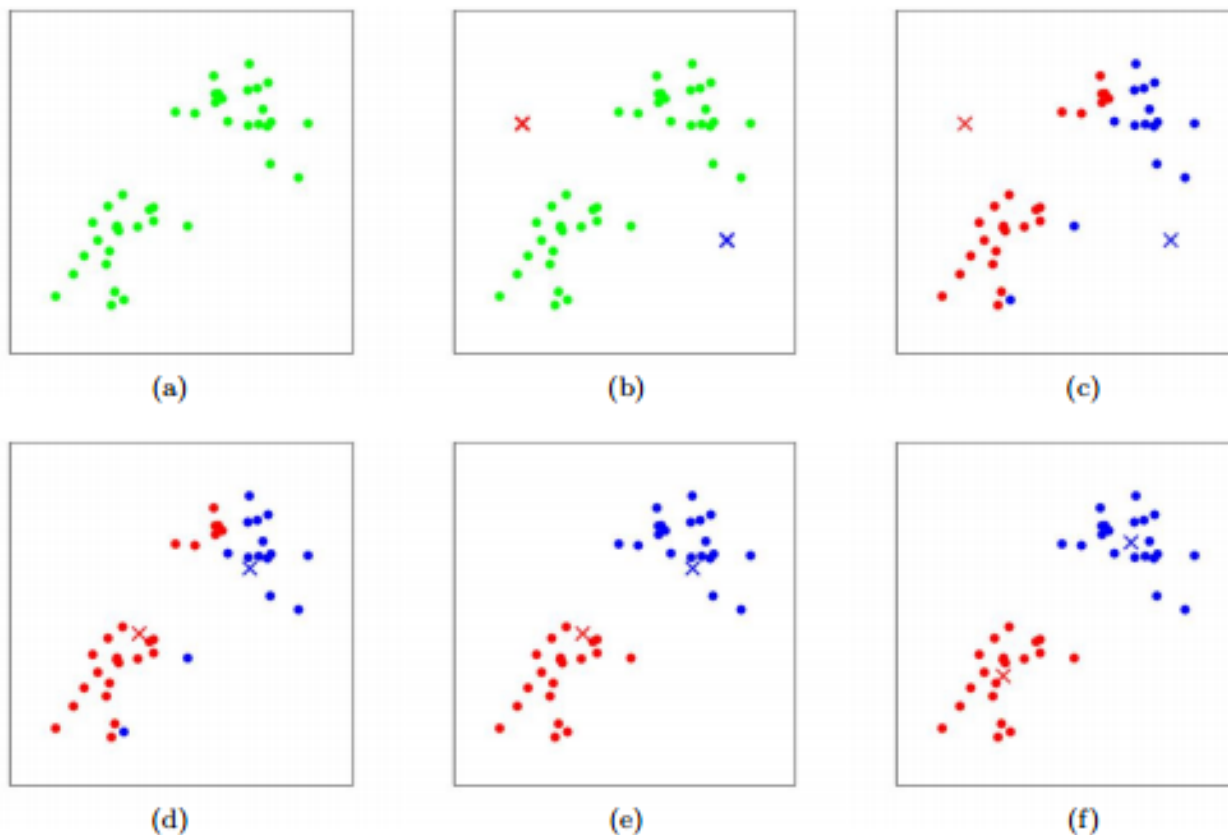
階層式分群（ Hierarchical Clustering ）：

```
# 這是由下而上的
from sklearn.cluster import AgglomerativeClustering
import numpy as np
# 載入資料
x = np.array([[1, 2], [1, 4], [1, 0],
              ...    [4, 2], [4, 4], [4, 0]])
# 演算法模型
clustering = AgglomerativeClustering().fit(X)
clustering
AgglomerativeClustering()
# 自動分成兩個群
clustering.labels_
array([1, 1, 1, 0, 0, 0])
```



pic from: towarddatascience.com

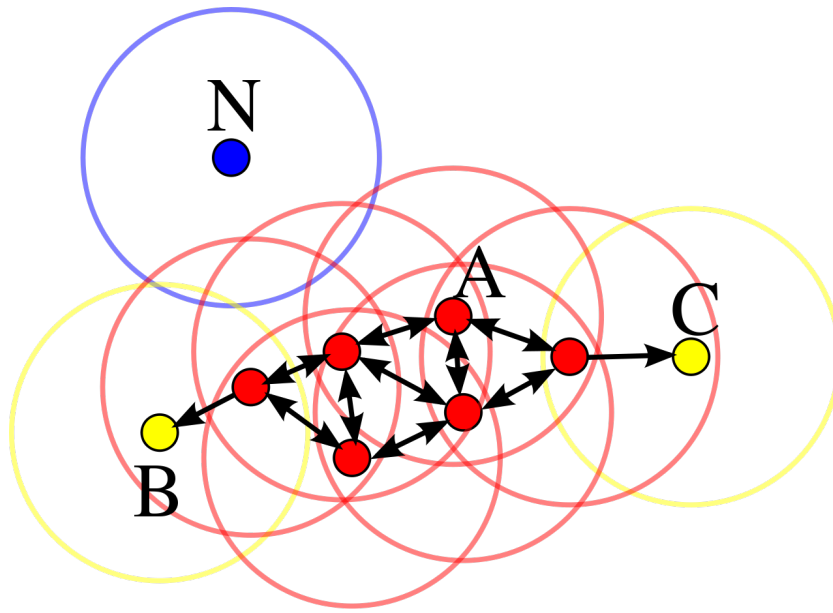
K-means Clustering簡介：



展示：Tibame_DataMiningAlgorithms.ipynb

```
from sklearn.cluster import KMeans
import numpy as np
#載入X
x = np.array([[1, 2], [1, 4], [1, 0],
              ... [10, 2], [10, 4], [10, 0]])
#k=2 k-mean演算法建模
kmeans = KMeans(n_clusters=2,
                 random_state=0).fit(X)
#分成k=2
kmeans.labels_
array([1, 1, 1, 0, 0, 0], dtype=int32)
#預測兩點
kmeans.predict([[0, 0], [12, 3]])
array([1, 0], dtype=int32)
#兩群中心在哪
kmeans.cluster_centers_
array([[10., 2.],
       [ 1., 2.]])
```

DBSCAN 簡介



```
from sklearn.cluster import DBSCAN
import numpy as np
x = np.array([[1, 2], [2, 2], [2, 3],
              ...      [8, 7], [8, 8], [25, 80]])
clustering = DBSCAN(eps=3, min_samples=2).fit(X)
clustering.labels_
array([ 0,  0,  0,  1,  1, -1])
clustering
DBSCAN(eps=3, min_samples=2)
```